Tugas Besar IF4031

Arsitektur Aplikasi Terdistribusi

ARLOW:

Agregasi Real-time Layanan Operasi Waspada

Laporan Milestone 1

Kelompok **nasigoreng**

Muhammad Aulia Azka	13523137
Arlow Emmanuel Hergara	13523161
Fachriza Ahmad Setiyono	13523162

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

10 Oktober 2026

Kontribusi Anggota

NIM	Nama	Kontribusi
13523137	Muhammad Aulia Azka	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*
13523161	Arlow Emmanuel Hergara	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*
13523162	Fachriza Ahmad Setiyono	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*

Kontribusi diisi manual oleh anggota agar pembagian perancangan, implementasi, dan penulisan

mencerminkan pekerjaan yang benar-benar dilakukan.

Daftar Isi

Kontribusi Anggota **i**

1 Gambaran Sistem **1**
1.1 Tujuan dan Ruang Lingkup Milestone 1 1
1.2 Arsitektur dan Batas Layanan 1
1.3 Teknologi dan Alasan Pemilihan 3
1.4 Asumsi dan Konfigurasi 4
1.5 Ringkasan Menjalankan Sistem 5
1.6 Strategi Verifikasi 5

2 P1: Interoperabilitas Data **6**
2.1 Rancangan Solusi 6
2.2 Justifikasi dan Alternatif 8
2.3 Batasan Solusi 8
2.4 Proof-of-Concept dan Keterlacakan 8

3 P2: Konkurensi dan Ketersediaan **13**
3.1 Rancangan Solusi 13
3.2 Justifikasi dan Alternatif 14
3.3 Batasan Solusi 14
3.4 Proof-of-Concept dan Keterlacakan 15

4 P3: Autentikasi dan Trust **22**
4.1 Rancangan Solusi 22
4.2 Justifikasi dan Alternatif 26
4.3 Batasan Solusi 26
4.4 Proof-of-Concept dan Keterlacakan 27

5 P4: Layanan dan Penyimpanan **35**
5.1 Rancangan Solusi 35
5.2 Justifikasi dan Alternatif 37
5.3 Batasan Solusi 38
5.4 Proof-of-Concept dan Keterlacakan 38

6 P5: Distribusi Event **43**
6.1 Rancangan Solusi 43
6.2 Justifikasi dan Alternatif 45
6.3 Batasan Solusi 46
6.4 Proof-of-Concept dan Keterlacakan 46

Daftar Pustaka **52**

Bab 1

Gambaran Sistem

1.1 Tujuan dan Ruang Lingkup Milestone 1

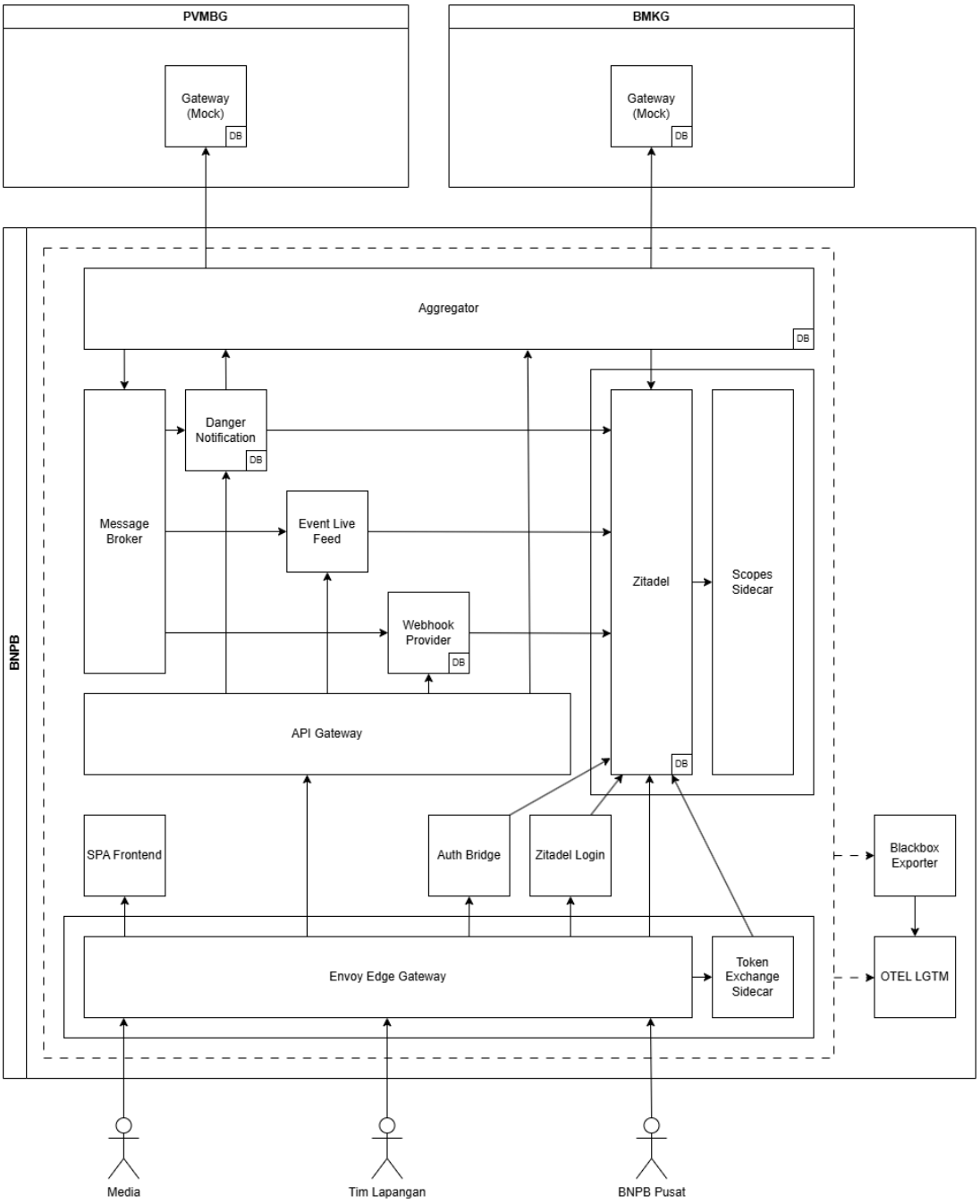
ARLOW: Agregasi Real-time Layanan Operasi Waspada mengintegrasikan kejadian gempa dan aktivitas gunung api dari mock BMKG dan PVMBG menjadi `HazardEvent` kanonik. BNPB mengambil data dengan polling, menyimpan hasil normalisasi, menyediakan API bagi klien, dan mendistribusikan publikasi yang sama kepada layanan live feed, webhook, serta notifikasi bahaya. Pengguna terdiri atas Media, Tim Lapangan, dan BNPB Pusat; hak akses ditentukan server berdasarkan identitas masing-masing.

Milestone ini berfokus pada interoperabilitas data (P1), isolasi kelambatan dan kegagalan sumber (P2), autentikasi dan otorisasi (P3), pemisahan layanan beserta penyimpanannya (P4), dan distribusi event melalui pub/sub (P5), sesuai spesifikasi tugas [1]. Data bencana adalah simulasi untuk pengujian, bukan informasi operasional kebencanaan. Klien membaca data hazard; operasi tulis terbatas pada pengaturan langganan, sesi, dan kontrol mock untuk demo.

Laporan diperbarui pada 10 Oktober 2026 berdasarkan snapshot repository `122ca7b`. Bukti manual P1–P5, dua keluaran k6, laporan suite otomatis, serta empat screenshot tambahan P3/P4 berasal dari commit `4a1973b`. Bukti baru memperlihatkan penolakan kredensial silang, refresh sesi OIDC pada browser, dan build Notification dengan `--no-deps`. Bukti historis tetap dicantumkan dengan tanggal aslinya. Penjelasan implementasi ditelusuri ke kode dan Compose pada snapshot `122ca7b`, yang menambahkan persistensi RabbitMQ, injeksi environment per layanan, perbaikan retry webhook, dan perbaikan pemetaan Aggregator setelah bukti diambil; pembaruan dokumen tidak menjalankan ulang pengujian. Cakupan bukti dijelaskan pada bab terkait; inventaris berkas tersedia di `docs/tex/evidence/manifest.json`.

1.2 Arsitektur dan Batas Layanan

Gambar 1.1 menunjukkan seluruh container pada stack utama beserta arah komunikasinya. Kotak BMKG dan PVMBG di bagian atas adalah sistem sumber di luar BNPB; masing-masing hanya diwakili gateway mock. Kotak putus-putus di dalam BNPB mengelompokkan container yang diinstrumentasi dan dipantau, sedangkan Blackbox Exporter dan OTEL LGTM berada di luarnya sebagai komponen observabilitas. Tanda *DB* menandakan penyimpanan milik container tersebut: SQLite privat pada mock, Webhook Provider, dan Danger Notification; PostgreSQL canonical store pada Aggregator; serta PostgreSQL Auth pada Zitadel. Kedua PostgreSQL berjalan sebagai container database terpisah dengan pemilik logis tunggal, bukan database bersama. Panah mengikuti arah pemanggil ke penerima, kecuali panah dari Message Broker yang mengikuti aliran publikasi.



Gambar 1.1: Arsitektur container ARLOW: Agregasi Real-time Layanan Operasi Waspada. Panah REST/gRPC mengikuti pemanggil ke penerima; panah dari Message Broker mengikuti aliran publikasi AMQP; panah putus-putus menunjukkan probe dan telemetri.

1. Media, Tim Lapangan, dan BNPB Pusat mengakses satu origin melalui Envoy Edge Gateway. Envoy meneruskan `/` ke SPA Frontend (Nginx), `/auth/*` ke Auth Bridge, halaman login ke Zitadel Login, serta request OIDC ke Zitadel. Tidak ada jalur langsung dari pengguna ke container lain.
2. Untuk setiap request `/api/`, Envoy memanggil Token Exchange Sidecar (Rust) melalui `ext_authz`. Sidecar menukar access token opaque milik pengguna dengan JWT ke Zitadel (RFC 8693), lalu Envoy meneruskan request beserta JWT tersebut ke API Gateway.
3. API Gateway (Go) menerjemahkan REST/JSON dan SSE menjadi pemanggilan gRPC ke Aggregator, Danger Notification, Event Live Feed, dan Webhook Provider. Pembacaan hazard mengambil canonical store melalui Aggregator, bukan memanggil mock secara sinkron.

4. Aggregator melakukan polling REST/JSON ke kedua mock dengan kredensial berbeda. Tidak ada push langsung dari mock ke BNPB. Hasil normalisasi dan outbox ditulis dalam satu transaksi PostgreSQL; publisher mengirim envelope Protobuf dari outbox ke Message Broker (RabbitMQ).
5. Message Broker membagikan publikasi ke queue tiap consumer. Event Live Feed menyediakan replay dan stream, Webhook Provider mengirim HTTP callback ke endpoint eksternal, dan Danger Notification menyimpan notifikasi serta push outbox sebelum worker mengirim melalui Firebase Cloud Messaging (FCM). Danger Notification juga memanggil Aggregator untuk status kesegaran sumber. Endpoint webhook dan FCM berada di luar sistem sehingga tidak digambarkan.
6. Aggregator, Danger Notification, Event Live Feed, dan Webhook Provider memverifikasi JWT secara mandiri memakai metadata dan kunci publik Zitadel. Role dan scope pada JWT dipasok Zitadel melalui Scopes Sidecar (`zitadel-claims`, target Actions v2).
7. Seluruh container di dalam kotak putus-putus mengirim trace, metric, dan log melalui OpenTelemetry ke OTEL LGTM. Blackbox Exporter melakukan probe HTTP/gRPC/TCP ke container tersebut dan hasilnya di-scrape oleh OTEL LGTM.

Kelompok komponen *Auth* pada rancangan awal direalisasikan oleh Zitadel, Zitadel Login, Auth Bridge, Token Exchange Sidecar, dan Scopes Sidecar. Auth Bridge hanya menangani `/auth/*` (OIDC dan sesi SQLite terenkripsi); permintaan API bisnis melewati Envoy dan API Gateway. Container sekali jalan `auth-bootstrap`, yang menyiapkan proyek, akun, grant, dan client OIDC sebelum layanan lain berjalan, tidak digambarkan karena tidak aktif saat runtime. Observabilitas menggunakan container `otel-lgtm` serta `blackbox`, bukan container Prometheus dan Jaeger terpisah seperti sketsa awal.

1.3 Teknologi dan Alasan Pemilihan

Komponen	**Teknologi**	**Alasan dan tanggung jawab**
BMKG dan PVMBG	Python 3.12, HTTP standard library, SQLite	Dependensi ringan; thread request memungkinkan health/admin tetap merespons saat request data menunggu. File database mempertahankan seed dan event setelah restart.
Frontend	React, TypeScript, Vite, Nginx	SPA dengan tampilan login, filter/daftar/detail hazard, status sumber, live feed, notifikasi, dan webhook; Nginx menyajikan hasil build statis.
Edge dan REST API	Envoy, Rust (Tonic); Go, chi, oapi-codegen	Envoy melakukan routing; sidecar Rust menukar token melalui `ext_authz`. Go menjaga bentuk JSON, status HTTP, validasi OpenAPI, dan framing SSE yang berbeda dari ProtoJSON.
Aggregator	Go, gRPC, PostgreSQL 17	Goroutine untuk polling independen; transaksi dan indeks relational untuk data inti, JSONB untuk atribut dinamis.
Event Live Feed	Rust, Tokio, Tonic	Stream asinkron dan buffer replay berbatas; API Gateway mengubahnya menjadi SSE untuk browser.
Webhook Provider	Go, SQLite WAL	Menyimpan subscription, hasil delivery, dan deduplikasi; worker mengirim HTTP callback bertanda tangan.

Komponen	Teknologi	Alasan dan tanggung jawab
Danger Notification	C#, ASP.NET gRPC, EF Core SQLite, Firebase Admin	Menyimpan notifikasi dan push outbox privat, menyediakan proyeksi summary/full, serta mengirim push FCM melalui worker. Status kesegaran sumber dibaca melalui Aggregator.
Identitas	Zitadel, PostgreSQL, Node.js 24, Rust	Provider mengelola akun; bridge menjalankan OIDC dan sesi SQLite terenkripsi; Rust Actions memasok role/scope.
Distribusi event	RabbitMQ 4, AMQP, Protobuf	Fanout dan queue independen memisahkan produsen dari consumer serta menyediakan acknowledgement.
Observabilitas	OpenTelemetry, otel-lgtm, blackbox exporter	Pengiriman trace/metric/log dan probe HTTP/gRPC/TCP; dashboard serta alert diprovisikan melalui berkas repo.
Orkestrasi	Docker Compose	Batas proses, jaringan, volume, dan dependensi dinyatakan di satu stack utama; Compose khusus dipakai untuk pengembangan parsial.

1.4 Asumsi dan Konfigurasi

1. Instalasi M1 berjalan pada satu host Docker. Port publik default adalah BNPB 8080, BMKG 8081, PVMBG 8082, Grafana 3000, serta RabbitMQ Management 15672. Port mock dalam container adalah 8080.
2. Host `localhost` dan `auth.localhost` menunjuk mesin demo. HTTP diizinkan secara eksplisit untuk lingkungan lokal; ini bukan konfigurasi TLS produksi.
3. Setiap mock menyiapkan minimal 20 record pada database baru. Interval event default 10 detik; polling contoh 3 detik. Semua nilai latensi konfigurasi merupakan parameter simulasi, bukan hasil benchmark.
4. ID sumber dan timestamp menjadi dasar watermark polling inklusif. Tabel referensi gunung statis merupakan data milik BNPB. Penerimaan field tambahan tidak menjamin kompatibilitas perubahan tipe atau penggantian nama field wajib.
5. Aggregator adalah pemilik logis canonical store untuk operasi bisnis. Layanan lain memakai gRPC atau event; Notification membaca notifikasi dari SQLite miliknya. Compose hanya memberikan DSN canonical store kepada Aggregator, tetapi collector juga mengakses PostgreSQL untuk metrik, database berada pada jaringan bersama, dan port PostgreSQL diekspos ke host; batas isolasi ini dijelaskan pada P4.
6. Akun pengguna di-seed pada Zitadel, bukan SQLite mock ataupun frontend. Tiga akun manusia memiliki role terpisah; service user untuk tes end-to-end memiliki kredensial tersendiri.
7. Notifikasi push memerlukan proyek Firebase dan service-account JSON lokal; konfigurasi web publik diperlukan untuk push browser. Tanpa dependensi tersebut, demo push belum dapat dianggap terbukti.
8. Restart mempertahankan volume bernama. Menghapus volume menghilangkan data persisten. Replay feed berada di memori, sedangkan state RabbitMQ disimpan pada volume `rabbitmq_data`.

1.5 Ringkasan Menjalankan Sistem

Prasyarat dasar adalah Docker Desktop/Engine yang aktif dan Python 3.12+ untuk helper setup. Siapkan konfigurasi lokal dari contoh; helper mempertahankan secret yang sudah tersedia. Helper mock hanya mengisi dua kredensial sumber, sehingga nilai `REPLACE_*` lainnya dalam `.env` harus diprovisikan sesuai pemiliknya. Isi kredensial broker, token internal yang digunakan, dan `AGGREGATOR_DB_PASSWORD`; DSN canonical store dibentuk Compose dari password tersebut. Sediakan file Firebase pada `FIREBASE_CREDENTIALS_PATH`. Nilai privat tidak dicantumkan dalam laporan.

```
python tools/setup_mock_env.py
python tools/setup_auth_env.py
docker compose --env-file .env --env-file .env.auth -f compose.yaml config --quiet
docker compose --env-file .env --env-file .env.auth -f compose.yaml up --build -d --wait
```

Perintah terakhir merupakan satu perintah orkestrasi setelah prasyarat konfigurasi dipenuhi. Buka `http://localhost:8080`, pilih masuk melalui Zitadel, lalu gunakan akun `media-demo`, `lapangan-demo`, atau `bnpb-demo`. Password berasal dari `.env.auth`. Dashboard Compose utama menggunakan `VITE_DATA_MODE=gateway`; fixture lokal hanya untuk mode pratinjau/stack Auth terpisah. Login UI milik Zitadel tidak mengubah frontend BNPB menjadi SSR.

Panduan operasional tersedia di `README.md`, `docs/mock-testing.md`, `docs/auth-testing.md`, `docs/e2e-testing.md`, dan `docs/load-testing.md`. Untuk layanan BNPB yang sudah berkembang, gunakan Compose dan README layanan terbaru bila uraian status pada README root masih tertinggal. Stack parsial `compose.mocks.yaml`, `compose.auth.yaml`, serta `compose.frontend.yaml` tetap berguna untuk pengembangan, tetapi bukan pengganti pengujian integrasi stack utama.

1.6 Strategi Verifikasi

Artefak pengujian tersimpan di `docs/tex/evidence/`, meliputi hasil tes mock, tes browser Auth, unit test Aggregator, ringkasan pengujian otomatis, serta JSON k6. Screenshot demonstrasi dikelompokkan pada direktori P1–P5 sesuai problem yang diuji. Path singkat seperti `P3/1-new.png` relatif terhadap direktori bukti tersebut; path berawalan `evidence/` relatif terhadap `docs/tex/`. Nama berkas dan hash setiap artefak tercantum pada `docs/tex/evidence/manifest.json`.

Bukti terdiri atas inspeksi implementasi, hasil tes otomatis tersimpan, tangkapan layar demo, dan pengukuran k6. Bab P1–P5 menyajikan hasil aktual beserta rujukan artefaknya. Hasil k6 mencatat beban 50 VU selama 60 detik dan isolasi latensi BMKG; screenshot RabbitMQ memperlihatkan fanout serta penumpukan dan pengurasan backlog consumer. Status PASS pada suite otomatis dilaporkan sebagai hasil suite dengan cakupan assertion yang dijalankan, bukan pengganti setiap observasi manual. Bukti browser Auth historis dibedakan dari pengujian token API terbaru. Gambar yang menampilkan nilai secret konfigurasi tidak direproduksi dalam PDF.

Bab 2

P1: Arsitektur dan Interoperabilitas Data

2.1 Rancangan Solusi

2.1.1 Kontrak Sumber dan Akuisisi Data

BMKG menyediakan `GET/seismic-events` dan `GET/tsunami-warnings`; PVMBG menyediakan `GET/volcanic-reports`. Ketiganya menerima `since` ISO 8601 yang inklusif dan mengembalikan koleksi `items`. Aggregator tidak meminta sumber mengikuti satu skema baru. Adapter per sumber memahami format asli, mengambil data dengan HTTP, lalu mengubahnya menjadi `HazardEvent`. Kontrak sumber didokumentasikan dalam `contracts/openapi/bmkg.yaml` dan `contracts/openapi/pvmbg.yaml`; kontrak keluaran publik ada pada `contracts/openapi/gateway.yaml`.

Kedua mock menggunakan SQLite file dengan kolom ID, waktu terindeks, dan payload JSON. Inisialisasi membuat tabel serta minimal 20 record ketika penyimpanan belum berisi data; seed tidak ditambahkan ulang pada setiap restart. Generator menambah satu event per interval, default 10 detik. BMKG memakai pola variasi berdasarkan indeks dan UUID untuk ID baru; tidak seluruh nilai gempa dibangkitkan acak. PVMBG memilih variasi laporan dengan bilangan acak, termasuk `confidence_level` ketika skema v2 aktif. Delay BMKG tetap, default 100 ms; delay PVMBG diambil acak antara 500–3000 ms. Sumber implementasi adalah `app.py`, `config.py`, dan `storage.py` pada direktori masing-masing mock.

2.1.2 Model Kanonik dan Pemetaan

`HazardEvent` memisahkan atribut lintas sumber dari rincian mentah. Tujuh field ringkasan adalah `hazard_id`, `source`, `hazard_type`, `severity`, `area_name`, `occurred_at`, dan `ingested_at`. Tampilan penuh menambahkan `source_ref_id`, `latitude`, `longitude`, serta `attributes`. BNPB memberikan ID kanonik dan waktu ingest saat menyimpan record. Unique key pasangan sumber dan ID asli menjaga identitas kanonik pada pengambilan berulang.

Field kanonik	BMKG	PVMBG
`source`	BMKG	PVMBG
`source_ref_id`	`event_id`	`report_id`
`hazard_type`	SEISMIC	VOLCANIC
`area_name`	`region_name`	Nama gunung dari referensi BNPB berdasarkan `volcano_id`.
Koordinat	`epicenter_lat`, `epicenter_lon`	Koordinat referensi gunung statis.
`occurred_at`	`occurred_at`	`reported_at`
`severity`	Tingkat warning terkait; jika tidak ada, berdasarkan magnitudo.	Normal/Waspada/Siaga/Awas menjadi enum huruf kapital.
`attributes`	Magnitudo, kedalaman, potensi tsunami, daftar warning, dan tambahan sumber.	Jumlah erupsi, tinggi kolom abu, ID gunung, confidence bila ada, dan tambahan sumber.

Tabel 2.1: Pemetaan ke model kanonik pada adapter Aggregator.

`BMKGPoller` mengambil gempa serta warning melalui endpoint terpisah, lalu membentuk indeks warning menurut `related_event_id`. Warning dihubungkan ke `event_id` sebelum normalisasi. Jika terdapat lebih dari satu warning, tingkat terberat dipilih dengan urutan AWAS, SIAGA, WASPADA. Tanpa warning, magnitudo $m < 5{,}0$ menghasilkan NORMAL, $5{,}0 \leq m < 6{,}5$ WASPADA, dan $m \geq 6{,}5$ SIAGA. Rincian warning disimpan sebagai array `attributes.tsunami_warnings`, sehingga informasi korelasi tetap tersedia pada proyeksi penuh.

Pemetaan PVMBG memakai `LookupVolcano` di `services/aggregator/internal/domain/volcano.go`. Implementasi menormalkan ID ke huruf kecil dan menerima awalan `v-`. Referensi memuat nama serta koordinat; dengan demikian mock tidak harus menambahkan field lokasi ke kontrak aslinya. Implementasi pemetaan kedua sumber berada di `services/aggregator/internal/ingest/mapper.go`.

2.1.3 Perubahan Skema saat Runtime

Mekanisme penanganan schema drift. Sistem menangani schema drift dengan pola *Tolerant Reader* [8] yang dipadukan dengan *attribute passthrough* dan penyimpanan *schema-on-read*. Ketiganya bekerja berlapis: (1) decoder adapter hanya mengikat field inti yang dibutuhkan untuk membentuk `HazardEvent` ke struktur bertipe dan tidak menolak payload karena field yang tidak dikenalnya; (2) field di luar kontrak yang dikenal diteruskan apa adanya ke `attributes`, bukan dibuang; (3) Canonical Store menyimpan `attributes` sebagai JSONB sehingga field baru tidak memerlukan migrasi DDL. Kebijakan ini disebut *evolusi skema aditif*: penambahan field diterima otomatis, sedangkan perubahan yang merusak kontrak inti tidak ditangani otomatis: perubahan tipe field inti membuat decoding gagal, sementara rename atau penghapusan field inti tidak terdeteksi sebagai drift dan menghasilkan nilai kosong. Keduanya memerlukan pembaruan adapter. Rincian setiap lapisan diuraikan berikut.

Operator mengirim `POST/admin/schema-version` dengan `{"version":2}` pada PVMBG. Pengaturan ini memengaruhi laporan yang dibuat selanjutnya, tanpa restart. Laporan lama tetap tidak memiliki `confidence_level`; laporan baru menyimpan nilai tersebut dalam JSON SQLite. Mengembalikan versi menjadi 1 tidak menghapus field yang telah tersimpan pada laporan v2.

Decoder Aggregator memiliki field `ConfidenceLevel` opsional sekaligus map `RawFields`. JSON dibaca menjadi struktur bertipe untuk field inti dan map untuk field tambahan. Field yang sudah dikenal dikeluarkan dari map, sedangkan field tidak dikenal diteruskan ke `attributes`. Atribut

hasil normalisasi, seperti `magnitude`, `volcano_id`, dan `threat_level`, ditulis setelah field tambahan sehingga field mentah bernama sama tidak dapat menimpanya. Kebijakan ini berlaku pada gempa, warning tsunami, dan laporan vulkanik. Nilai confidence yang tidak ada dibedakan dari nilai numerik 0. PostgreSQL menyimpan atribut dalam JSONB tanpa menambah kolom; kontrak internal membawa atribut melalui struktur dinamis Protobuf. Pemanfaatan JSONB ini sesuai kemampuan penyimpanan dan pemrosesan JSON pada PostgreSQL [6].

2.2 Justifikasi dan Alternatif

Model kanonik menghindarkan klien dari keharusan memahami dua format, dua kredensial sumber, serta aturan korelasi tsunami. Adapter terpisah membuat perubahan pada satu sumber tidak harus merombak API publik. Alternatif meneruskan payload mentah apa adanya lebih singkat untuk dibuat, tetapi setiap klien harus menduplikasi pemetaan dan pemisahan akses Media.

Tolerant Reader—struktur inti bertipe dengan map tambahan—dipilih sebagai kompromi. Deserialisasi sangat ketat yang menolak semua field baru dapat menghentikan ingest ketika confidence mulai muncul. Sebaliknya, map bebas untuk seluruh record memindahkan kesalahan tipe dan field wajib ke tahap yang lebih jauh. Implementasi saat ini mempertahankan tipe inti dan menggunakan map untuk perubahan aditif. Field tambahan dapat ditelusuri kembali dalam proyeksi penuh, tanpa memperluas tujuh field ringkasan Media.

Polling mengikuti spesifikasi dan memudahkan pengujian sumber yang tidak menyediakan push. Watermark mengurangi pengambilan data berulang, sementara upsert melindungi identitas record. Konsekuensinya, data yang terlihat klien tertinggal setidaknya selama waktu tunggu polling dan pemrosesan; sistem ini bukan integrasi real-time dengan jaminan keterlambatan nol.

2.3 Batasan Solusi

- Dukungan perubahan aditif tidak otomatis menangani rename, penghapusan field wajib, atau perubahan tipe, misalnya `magnitude` menjadi objek. Perubahan semantik tersebut memerlukan pembaruan adapter dan kontrak.
- Field yang tidak dikenal diteruskan, tetapi tidak otomatis dipahami secara bisnis.
- JSONB tidak menghilangkan kebutuhan validasi domain. Data tidak lengkap dan enum tidak dikenal belum memiliki karantina ingest yang menyeluruh.

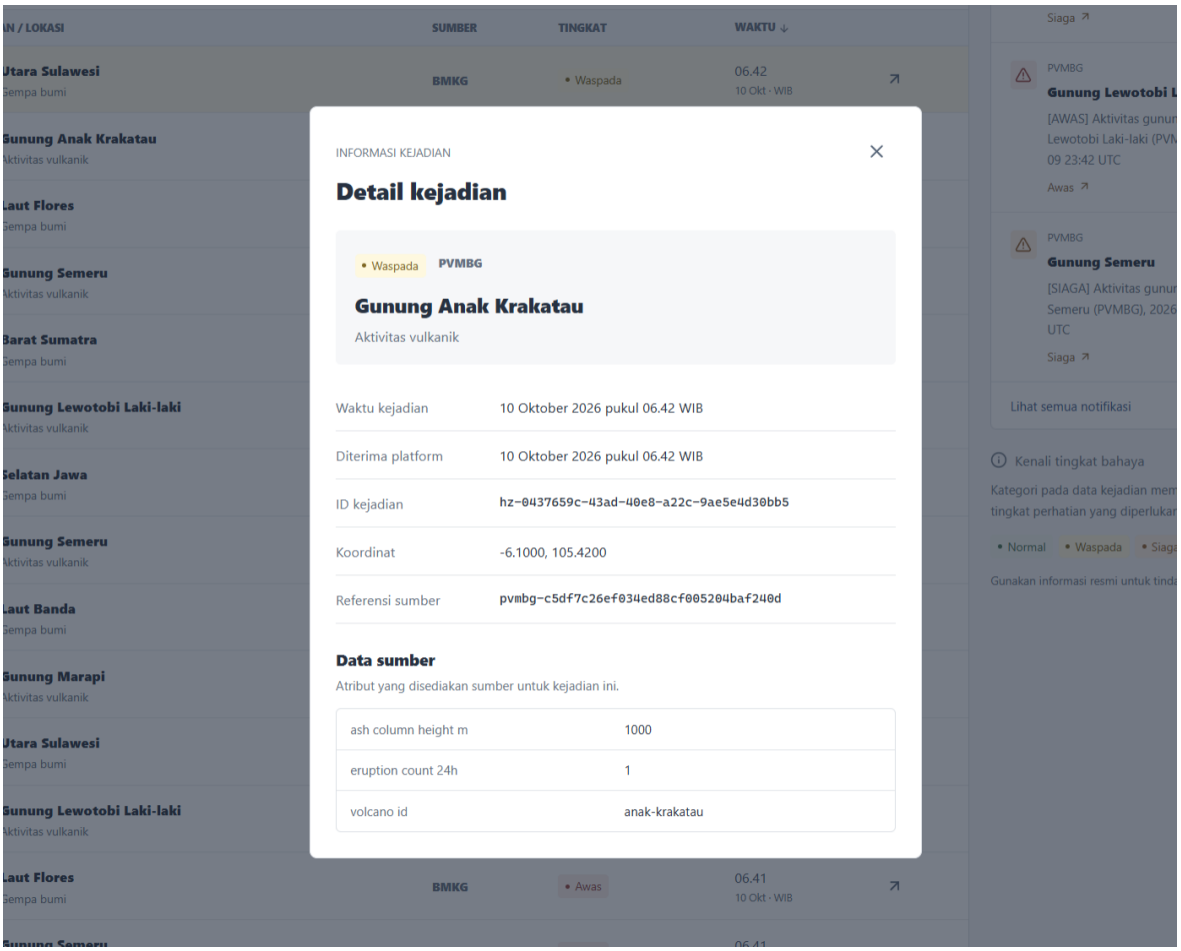
2.4 Proof-of-Concept dan Keterlacakan

Bukti P1 terdiri atas hasil tes unit Aggregator dalam `aggregator-unit-2026-10-09.jsonl`, laporan pengujian stack dalam `problem_set_report.md`, dan tangkapan layar demonstrasi P1 yang diuraikan pada bagian ini. Hasil tes unit Aggregator merekam tujuh tes tingkat utama paket ingest yang lulus pada 9 Oktober 2026, mencakup fetch HTTP, pemetaan gempa/warning, laporan vulkanik, evolusi skema, serta proyeksi. Paket domain tidak mempunyai test case dan tidak dihitung sebagai tes lulus. Laporan pengujian stack, yang dibuat pada 10 Oktober 2026, mencatat lima tes P1 lulus, tanpa error/failure maupun skip, dengan durasi 2,42 detik.

2.4.1 Hasil Demonstrasi Manual

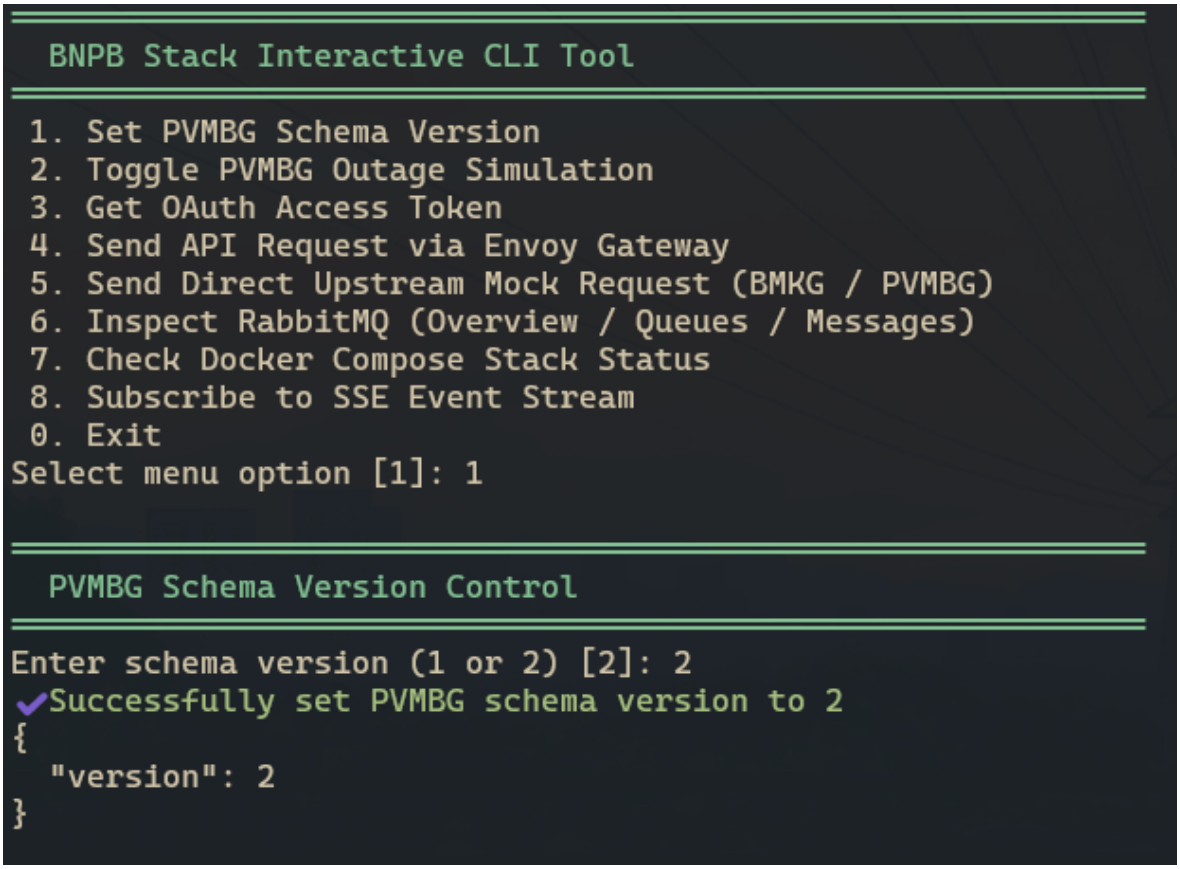
Pada kondisi awal, tangkapan layar demonstrasi P1 memperlihatkan detail BMKG Selatan Jawa dengan magnitudo 5,6, kedalaman 20 km, dan severity WASPADA. Detail PVMBG Gunung Anak Krakatau menampilkan ID kanonik, referensi sumber, koordinat, jumlah erupsi, serta

tinggi kolom abu tanpa atribut confidence. Kedua tampilan menggunakan bentuk detail kejadian yang sama meskipun rincian sumbernya berbeda. Nilai magnitudo dan severity BMKG pada contoh ini konsisten dengan aturan pemetaan ketika tidak terdapat warning tsunami terkait.

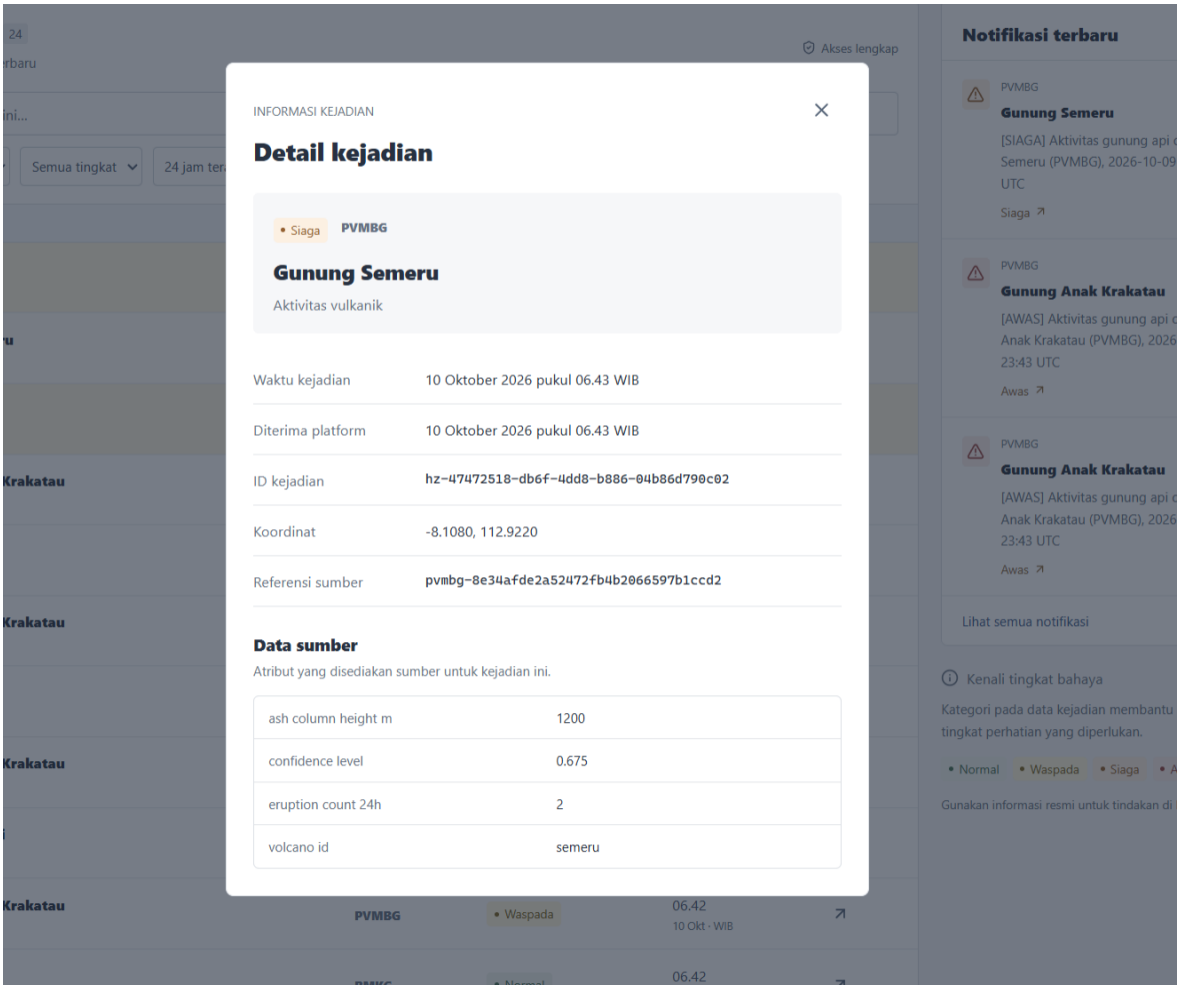


Gambar 2.1: Detail PVMBG sebelum perubahan skema; atribut sumber belum memuat confidence. Sumber: `P1/1-2.png`.

Operator kemudian mengubah versi PVMBG menjadi 2 melalui menu kontrol yang memanggil endpoint admin; respons yang direkam adalah `{"version": 2}`. Sesudah perubahan, antarmuka tetap menampilkan kejadian dari kedua sumber. Detail BMKG Barat Sumatra masih memuat magnitudo 5,6 dan severity WASPADA; detail PVMBG Gunung Semeru pada pukul 06.43 WIB tanggal 10 Oktober 2026 memuat `confidence_level` bernilai 0,675, selain atribut vulkanik yang sebelumnya sudah ada. Urutan ini memperlihatkan bahwa field tambahan diteruskan sampai tampilan klien BNPB.



Gambar 2.2: Respons kontrol runtime ketika versi skema PVMBG diubah menjadi 2. Sumber: `P1/2.png`.



Gambar 2.3: Detail PVMBG sesudah perubahan; confidence bernilai 0,675 tampil bersama atribut vulkanik lainnya. Sumber: `P1/3-2.png`.

2.4.2 Cakupan Bukti

Kriteria	Bukti tersedia	Kesimpulan dan batas cakupan
Pemetaan dua sumber	Tes unit Aggregator; tes awal pada laporan stack; `P1/1-1.png` dan `P1/1-2.png`.	Struktur kanonik dan contoh nilai kedua sumber terlihat. Tes unit melengkapi pemeriksaan aturan korelasi warning.
Perubahan v1 ke v2	`P1/2.png`, `P1/3-1.png`, dan `P1/3-2.png`; tes perubahan skema pada laporan stack.	Kontrol versi berhasil dan kedua sumber tetap tampil; confidence terlihat pada hasil PVMBG.
Field tambahan pada attributes	Tes unit evolusi skema Aggregator; tes penerusan field pada laporan stack; detail v2 pada `P1/3-2.png`.	Kebijakan aditif teramati untuk confidence; tidak menyatakan semua jenis perubahan skema didukung.
Koeksistensi record v1/v2	Laporan stack, tes `test_05_v1_and_v2_coexistence_in_canonical_store`.	Tes memeriksa record tanpa dan dengan confidence dalam hasil query yang sama. Tangkapan layar demonstrasi P1 menunjukkan contoh sebelum dan sesudah secara terpisah.
Kontinuitas tanpa restart	Kode kontrol runtime dan urutan tangkapan layar demonstrasi P1.	Artefak tidak menyertakan pasangan ID/start time container sebelum–sesudah; kontinuitas proses belum diukur langsung.

Tabel 2.2: Keterlacakan P1 terhadap bukti unit, otomatis, dan manual.

Laporan pengujian stack memperkuat demonstrasi manual, dengan batas metode bahwa pencarian record v2 pada tes menerima record yang sudah memiliki confidence dan tidak mengikatnya pada ID serta waktu sesudah toggle. Oleh karena itu, laporan membedakan keberhasilan membaca data v1/v2 dari pembuktian bahwa record tertentu baru dihasilkan pada percobaan tersebut. Respons mentah sebelum–sesudah serta snapshot container akan memperkuat keterlacakan pada pengujian berikutnya; tangkapan layar demonstrasi P1 yang tersedia sudah memperlihatkan hasil perubahan aditif pada antarmuka.

Bab 3

P2: Konkurensi dan Ketersediaan

3.1 Rancangan Solusi

Sistem memisahkan pengambilan data sumber dari pelayanan permintaan pengguna. Aggregator menjalankan poller BMKG dan PVMBG sebagai dua goroutine independen, sementara layanan gRPC membaca data kanonis di PostgreSQL. Publisher outbox berjalan pada goroutine lain. Pemisahan ini terlihat pada `services/aggregator/cmd/main.go`. Dengan demikian, keterlambatan PVMBG memengaruhi waktu pembaruan data PVMBG, bukan menambahkan waktu tunggu upstream ke setiap pembacaan BMKG. Permintaan pengguna tetap bergantung pada kapasitas gateway, autentikasi, Aggregator, dan database yang dipakai bersama.

Setiap poller melakukan pengambilan awal, kemudian mengulangi polling menurut interval masing-masing. HTTP client memiliki timeout per sumber dan menerima context untuk pembatalan saat layanan berhenti. Dalam satu poller, satu putaran diselesaikan sebelum putaran berikutnya diproses; implementasi tidak membuat goroutine baru tanpa batas pada setiap tick. BMKG mengambil seismic event lalu peringatan tsunami secara berurutan, sedangkan PVMBG mengambil laporan vulkanik. Implementasi berada pada `services/aggregator/internal/ingest/bmkg.go` dan `services/aggregator/internal/ingest/pvmbg.go`.

Tabel 3.1: Parameter konkurensi dan kesegaran data

Parameter	**Nilai acuan**	**Makna**
Interval polling kedua sumber	3.000 ms	Contoh konfigurasi di `.env.example`.
Timeout BMKG / PVMBG	1.000 / 3.500 ms	Batas setiap request HTTP upstream, bukan target latensi API.
Ambang stale	10.000 ms	Acuan usia keberhasilan pengambilan sumber.
Timeout unary gateway	2.000 ms	Default deadline panggilan backend.
Permintaan unary aktif	100	Default kapasitas semaphore gateway; stream dikecualikan.
Koneksi PostgreSQL	5–25	Minimum dan maksimum pool Aggregator.

Setiap sumber mempunyai watermark, waktu keberhasilan terakhir, waktu percobaan terakhir, dan pesan kegagalan. Setelah fetch berhasil, data dinormalisasi dan disimpan sebelum watermark dimajukan. Pada kegagalan, data sebelumnya tidak dihapus dan watermark keberhasilan dipertahankan. Pada pembacaan API, `services/aggregator/internal/domain/status.go` menghitung status berikut:

- **FRESH**: keberhasilan terakhir masih dalam ambang waktu dan tidak ada error sumber yang

tercatat.
- **STALE**: sumber gagal atau terlalu lama tidak berhasil, tetapi data kanonis sumber tersebut masih tersedia.
- **UNAVAILABLE**: tidak tersedia data tersimpan untuk digunakan ketika sumber belum berhasil atau tidak dapat diperbarui.

Respons hazard tetap membawa item yang tersedia beserta metadata `freshness`. Nilai `partial=true` diberikan apabila sumber yang relevan dengan permintaan tidak fresh. Filter sumber membatasi pemeriksaan kesegaran pada sumber tersebut. Mekanisme ini membedakan kondisi tidak ada peristiwa dari sumber yang belum dapat memberikan data, sekaligus memungkinkan klien menampilkan usia data. Pembentukan respons ada pada `services/aggregator/internal/grpc/server.go`; penyimpanan watermark ada pada `services/aggregator/internal/store/hazard_store.go`.

Gateway juga membatasi permintaan unary aktif dengan channel berkapasitas tetap. Ketika penuh, permintaan ditolak dengan respons overload sesuai kontrak, umumnya HTTP 429, dan `Retry-After`. Deadline unary membatasi waktu tunggu backend. Pembatas ini bukan pembatas laju per pengguna dan tidak menghitung stream SSE yang berumur panjang. Kode terkait adalah `services/api-gateway/internal/server/server.go`, `services/api-gateway/internal/config/config.go`, dan `services/api-gateway/internal/problem/problem.go`.

3.2 Justifikasi dan Alternatif

Polling independen dipilih karena data kebencanaan dikumpulkan secara berkala dan dapat dilayani dari penyimpanan kanonis. Pengguna memperoleh respons dari satu bentuk data meskipun kedua sumber memiliki latensi serta skema berbeda. Biaya keputusan ini adalah jeda antara perubahan upstream dan kemunculannya pada API. Karena itu, freshness merupakan bagian dari kontrak respons, bukan sekadar informasi log.

Alternatif pertama ialah melakukan dua panggilan upstream secara serial pada setiap request. Pendekatan tersebut sederhana, tetapi latensi sumber terakumulasi dan gangguan satu sumber langsung memengaruhi request pengguna. Alternatif kedua ialah fan-out paralel per request dengan deadline terpisah. Pendekatan kedua mengurangi penjumlahan waktu tunggu, tetapi tetap menambah beban upstream sebanding dengan jumlah pengguna dan memerlukan fallback tersendiri.

Penyimpanan kanonis dengan polling mengurangi keterikatan tersebut, dengan konsekuensi kapasitas database menjadi sumber daya bersama. Pool koneksi dan batas unary gateway menahan pertumbuhan pekerjaan aktif. Penggunaan circuit breaker atau penjadwal polling terdistribusi dapat menjadi pengembangan berikutnya, tetapi keduanya belum menjadi mekanisme yang digunakan pada implementasi ini.

3.3 Batasan Solusi

Isolasi pada tingkat goroutine tidak menjamin isolasi CPU, memori, database, atau autentikasi. Permintaan BMKG dan PVMBG tetap melewati komponen bersama. Kegagalan PostgreSQL tidak dapat ditutupi oleh fallback memori karena jalur baca kanonis masih membutuhkan database. Compose juga belum memberikan bukti failover beberapa replika Aggregator atau database.

Pembaruan watermark dilakukan sesudah transaksi data selesai, bukan di dalam transaksi yang sama. Restart pada sela kedua operasi dapat menyebabkan data diambil kembali; kestabilan identitas hazard menjaga baris kanonis tetap satu per referensi sumber.

Nilai timeout, kapasitas pool, serta semaphore merupakan konfigurasi atau mekanisme yang ditemukan pada kode. Hasil beban yang dilampirkan di bawah mengukur satu lingkungan lokal dan satu jendela pengujian; hasil tersebut tidak merupakan jaminan kapasitas produksi atau availability jangka panjang. Skenario outage membuktikan degradasi dan pemulihan pada demonstrasi yang dicatat, bukan failover infrastruktur.

3.4 Proof-of-Concept dan Keterlacakan

Bukti historis container pada 30 September 2026 disimpan dalam `docs/tex/evidence/mock-container-2026-09-30.json`. Catatan tersebut memuat 10 pemeriksaan berhasil pada lingkup mock. Pengukuran langsung BMKG ketika empat request PVMBG berjalan paralel tercatat 113,86 ms; pemeriksaan health dan admin PVMBG tercatat masing-masing 22,29 ms dan 17,80 ms. Angka ini menunjukkan perilaku pada skenario mock yang dicatat, bukan p95 gateway atau hasil 50 pengguna selama 60 detik.

Tabel 3.2: Status pembuktian P2 berdasarkan bukti yang dilampirkan

Skenario	**Target atau cakupan**	**Hasil**
Konkurensi mock	Request BMKG bersamaan dengan empat request PVMBG	Bukti historis tersedia.
Beban API lengkap	50 VU selama 60 detik; error tidak terkendali di bawah 1%	Terpenuhi: error 0%, p95 243,96 ms (hasil k6 beban; `P2/2.png`).
Isolasi melalui gateway	Target delay PVMBG 3 detik; p95 grup BMKG di bawah 300 ms	p95 87,09 ms; probe delay aktual sekitar 2,92 detik (hasil k6 isolasi; `P2/1.png`).
Degradasi sumber	Data tersimpan tetap dilayani dan status kembali fresh sesudah outage dinonaktifkan	HTTP 200, PVMBG stale, lalu pulih (demo outage; laporan pengujian otomatis).

3.4.1 Tooling dan Konfigurasi Load Testing

Pengujian beban menggunakan **Grafana k6** [9]. k6 dipilih karena skenario ditulis sebagai skrip JavaScript yang tersimpan di repositori, mendukung beberapa *scenario* dengan executor dan jadwal mulai berbeda dalam satu run, serta dapat memberi tag pada setiap request sehingga persentil dapat dihitung per fase dan per endpoint. Dua skrip digunakan: `tests/load/aggregator.js` untuk beban berkelanjutan dan `tests/load/aggregator-isolation.js` untuk isolasi latensi; fungsi bersama (token, request, validasi, ringkasan) berada pada `tests/load/lib.js`. Petunjuk menjalankan tersedia pada `docs/load-testing.md`.

Jalur yang diuji. k6 tidak memanggil Aggregator secara langsung. Setiap request dikirim ke Envoy pada `http://127.0.0.1:8080` dengan header `Host:localhost:8080`, sehingga melewati jalur yang sama dengan klien: Envoy → `ext_authz` Token Exchange Sidecar → API Gateway → gRPC Aggregator → PostgreSQL. Identitas yang dipakai adalah akun service `svc-bnpb` (role BNPB_INTERNAL); secret-nya dibaca dari `.cache/auth/`.

Alur setiap Virtual User (VU).

1. **Setup (sekali per run).** Skrip meminta token lalu memanggil `GET/api/v1/hazards?page_size=100` sampai daftar tidak kosong, kemudian menyimpan hingga 100 `hazard_id` untuk request detail. Request setup diberi tag tersendiri dan tidak dihitung dalam hasil.
2. **Token per VU.** Setiap VU meminta access token sendiri melalui `POST/oauth/v2/token` (grant `client_credentials`, Host `auth.localhost:8080`) dan menyimpannya di memori VU. Token diperbarui 10 detik sebelum TTL 60 detik berakhir, sehingga selama 60 detik fase beban tiap VU melakukan sekitar satu sampai dua kali penerbitan token. Request token diberi tag `kind:token` dan dikeluarkan dari persentil API; kegagalannya dihitung pada metrik `token_failures`.
3. **Iterasi.** Setiap iterasi mengirim tepat satu request API dengan header `Authorization: Bearer` dan `X-Correlation-ID` unik (`runId-vu-iterasi`) untuk pencocokan trace. Tidak ada *think time*; VU langsung memulai iterasi berikutnya setelah respons diterima (model *closed-loop*), sehingga 50 VU berarti paling banyak 50 request API bersamaan.
4. **Validasi respons.** Request dianggap error bila status bukan 200 atau bentuk body tidak sesuai: daftar wajib memiliki array `items`, detail wajib memiliki `item.hazard_id`, status sumber wajib memiliki array `freshness`. Jumlah per status HTTP (termasuk 429 dan error jaringan) dicatat terpisah.

Tabel 3.3: Campuran request per iterasi pada skrip k6

Skrip / grup	**Peluang**	**Endpoint yang dipanggil**
`aggregator.js`	60%	`GET/api/v1/hazards?page_size=20`, bergiliran tanpa filter, dengan parameter `source=BMKG`, dan dengan `source=PVMBG`.
	30%	`GET/api/v1/hazards/<hazard_id>` dengan ID acak dari hasil setup.
	10%	`GET/api/v1/hazards/sources/statuses`.
Isolasi, grup BMKG	40%	Daftar `page_size=20` dengan `source=BMKG`.
	30%	Daftar `page_size=20` dengan `hazard_type=SEISMIC`.
	30%	Detail hazard BMKG acak.
Isolasi, grup PVMBG	40/30/30%	Pola sama dengan `source=PVMBG`, `hazard_type=VOLCANIC`, dan detail hazard PVMBG.

Tabel 3.4: Scenario k6 dan executor yang dipakai

Scenario	**Executor**	**Jadwal dan beban**
`warmup` (beban)	`ramping-vus`	1 → 50 VU selama 10 detik; membuka koneksi dan mengisi cache, tidak dihitung.
`load`	`constant-vus`	50 VU selama 60 detik, mulai detik ke-10; satu-satunya jendela yang dilaporkan.
`warmup` (isolasi)	`constant-vus`	20 VU BMKG selama 5 detik.
`baseline` + `baseline_filler`	`constant-vus`	10 VU BMKG terukur + 10 VU BMKG pengisi selama 30 detik.
`contention_bmkg` + `contention_pvmbg`	`constant-vus`	10 VU BMKG terukur + 10 VU PVMBG selama 60 detik, mulai setelah baseline.

Grup kedua pada skrip isolasi sengaja menjaga total VU tetap 20 pada kedua fase; satu-satunya perbedaan baseline dan kontensi adalah jenis lalu lintas grup kedua (BMKG → PVMBG). Threshold yang menentukan lulus/gagal adalah `p(95)<300` ms dan error di bawah 1% pada request BMKG fase kontensi; pada skrip beban, error API di bawah 1% dan tidak ada kegagalan

token.

Prosedur pengulangan:

1. Jalankan stack yang telah diprovisikan, pastikan akun layanan tersedia pada `.cache/auth/`, dan buat direktori `.cache/load/`. Jalankan `k6runtests/load/aggregator.js` dari root repositori. Parameter dapat diubah melalui `LOAD_VUS`, `LOAD_DURATION`, dan `LOAD_RAMP`.
2. Untuk isolasi, tetapkan `PVMBG_RESPONSE_DELAY_MIN_MS` dan `PVMBG_RESPONSE_DELAY_MAX_MS` menjadi 3000, lalu buat ulang `pvmbg-mock` sebagaimana panduan. Jalankan `k6 runtests/load/aggregator-isolation.js`. Setup mengirim tiga request langsung ke `/volcanic-reports` pada mock (port 8082) dan membatalkan pengujian bila median delay kurang dari 2900 ms.
3. Simpan keluaran terminal dan ringkasan JSON yang ditulis skrip ke `.cache/load/`. Ringkasan isolasi mencantumkan jendela waktu baseline dan kontensi untuk pencocokan trace. Pulihkan konfigurasi delay setelah selesai.

Latensi yang diukur skrip tersebut mencakup Envoy, token exchange, gateway, RPC, dan pembacaan database. Angka yang dilaporkan merupakan latensi jalur API lengkap, bukan waktu eksekusi Aggregator saja. Hasil k6 beban tersimpan pada `evidence/P2/aggregator-2026-10-09T15-31-17-502Z.json`; hasil k6 isolasi tersimpan pada `evidence/P2/aggregator-isolation-2026-10-09T15-37-25-357Z.json`, relatif terhadap direktori laporan. Tangkapan layar pada direktori `evidence/P2/` memperlihatkan konfigurasi, fase, hasil, dan nama ringkasan JSON yang bersesuaian.

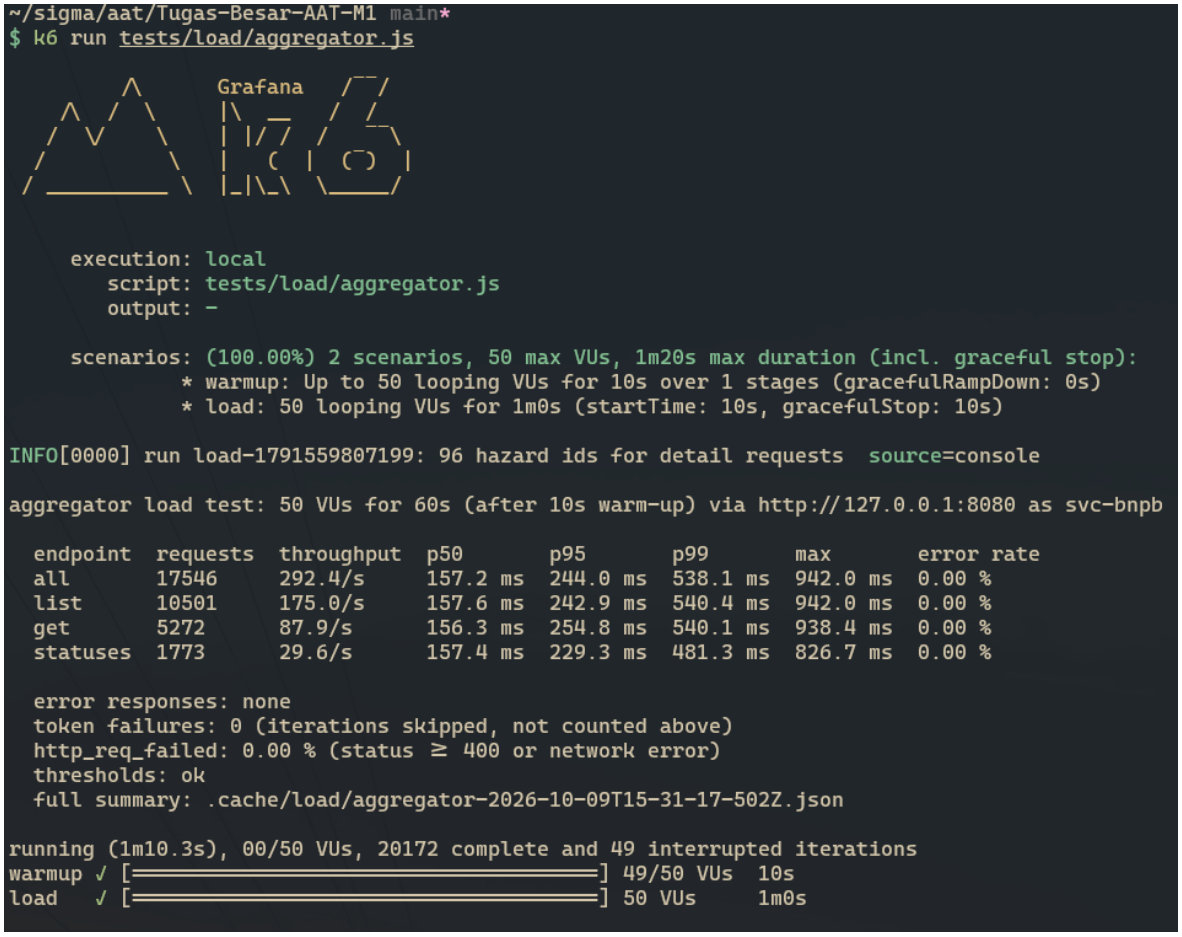
Untuk menilai batas error spesifikasi, HTTP 429 yang merupakan backpressure terkendali dilaporkan terpisah dari error tidak terkendali. Pada fase beban, catat jumlah request API N, jumlah 429 N_{429}, dan kegagalan lain N_{err}; laporkan $100N_{\text{err}}/N$ serta $100N_{429}/N$. Denominator dan definisi kegagalan harus dicantumkan agar angka dapat diperiksa. Threshold error bawaan skrip masih menghitung 429, sehingga lebih ketat daripada penghitungan penerimaan yang mengecualikannya. Hasil token endpoint tidak dicampur ke persentil request API.

3.4.2 Beban Berkelanjutan dan Isolasi Latensi

Pengujian k6 beban pada 9 Oktober 2026 menjalankan 50 VU selama 60 detik setelah pemanasan 10 detik. Fase beban mencatat 17.546 request API: 10.501 daftar, 5.272 detail, dan 1.773 status sumber. Tidak ada kegagalan API, HTTP 429, atau kegagalan pengambilan token pada fase tersebut. Throughput fase beban adalah $17\,546/60 = 292{,}43$ request/detik. Nilai `rate` bawaan pada ringkasan JSON menggunakan durasi seluruh run, termasuk pemanasan, sehingga tidak digunakan sebagai throughput jendela beban 60 detik.

Tabel 3.5: Hasil pengukuran P2; latensi dalam milidetik

Metrik	**Beban 50 VU / 60 detik**	**BMKG saat kontensi / 60 detik**
Jumlah request	17.546	9.407
Throughput (request/detik)	292,43	156,78
p50	157,19	59,50
p95	243,96	87,09
p99	538,14	126,99
Error tidak terkendali	0 / 17.546 (0%)	0 / 9.407 (0%)
HTTP 429	0	0



Gambar 3.1: Hasil k6: pemanasan 10 detik diikuti 50 VU selama 60 detik; fase beban mencatat 17.546 request tanpa error (sumber: `P2/2.png`).

Pengujian k6 isolasi menggunakan 10 VU BMKG yang diukur dan 10 VU kelompok kedua. Pada baseline 30 detik, kelompok kedua juga meminta BMKG; pada fase kontensi 60 detik, kelompok kedua meminta PVMBG. Probe langsung ke mock mencatat delay 2.921,29; 2.921,80; dan 2.928,30 ms. Nilai aktual ini memenuhi ambang validasi skrip 2.900 ms dan mendekati target konfigurasi 3.000 ms; hasil pengukuran tidak dinyatakan tepat 3.000 ms.

Pada baseline, p95 kelompok BMKG adalah 100,87 ms. Pada fase kontensi, p95 BMKG menjadi 87,09 ms dari 9.407 request, memenuhi ambang 300 ms; kelompok PVMBG menjalankan 9.382 request tanpa error. Kedua fase melewati jalur API publik yang sama. Perbedaan p95 sebesar sekitar −13,78 ms merupakan hasil run ini dan tidak ditafsirkan sebagai jaminan bahwa tambahan beban PVMBG selalu mempercepat BMKG.

```
~/sigma/aat/Tugas-Besar-AAT-M1 main*
$ k6 run tests/load/aggregator-isolation.js

        /\      Grafana   /‾‾/
   /\  /  \     |\  __   /  /
  /  \/    \    | |/ /  /   ‾‾\
 /          \   |   (  |  (‾)  |
/ __________ \  |_|\_\  \_____/


     execution: local
        script: tests/load/aggregator-isolation.js
        output: -

     scenarios: (100.00%) 5 scenarios, 20 max VUs, 1m45s max duration (incl. graceful stop):
              * warmup: 20 looping VUs for 5s (exec: bmkg)
              * baseline: 10 looping VUs for 30s (exec: bmkg, startTime: 5s)
              * baseline_filler: 10 looping VUs for 30s (exec: bmkg, startTime: 5s)
              * contention_bmkg: 10 looping VUs for 1m0s (exec: bmkg, startTime: 35s, gracefulStop: 10s)
              * contention_pvmbg: 10 looping VUs for 1m0s (exec: pvmbg, startTime: 35s, gracefulStop: 10s)

INFO[0008] run iso-1791560150226: PVMBG mock delay 2921/2922/2928 ms; 83 BMKG and 83 PVMBG hazard ids  source=console

BMKG/PVMBG isolation test via http://127.0.0.1:8080 as svc-bnpb
  config: 10 measured BMKG VUs + 10 second-group VUs (BMKG in baseline, PVMBG in contention); warm-up 5s, baseline 30s, contention 60s
  PVMBG mock delay (probe, 3 requests): min 2921 ms, median 2922 ms, max 2928 ms
  baseline window:   2026-10-09T15:35:55.287Z – 2026-10-09T15:36:25.287Z
  contention window: 2026-10-09T15:36:25.287Z – 2026-10-09T15:37:25.287Z

  phase                  requests  throughput  p50      p95       p99       max       error rate
  baseline BMKG          4382      146.1/s     62.4 ms  100.9 ms  161.2 ms  574.0 ms  0.00 %
  baseline 2nd group BMKG  4366      145.5/s     62.4 ms  101.4 ms  162.5 ms  575.2 ms  0.00 %
  contention BMKG        9407      156.8/s     59.5 ms  87.1 ms   127.0 ms  658.6 ms  0.00 %
  contention PVMBG       9382      156.4/s     59.8 ms  87.3 ms   129.6 ms  641.7 ms  0.00 %

  PASS: BMKG-only p95 under PVMBG contention = 87.1 ms (target < 300 ms, 9407 samples)
  BMKG p95 change vs baseline: -13.8 ms
  contention BMKG errors: none
  contention PVMBG errors: none
  token failures: 0 (iterations skipped, not counted above)
  thresholds: ok
  full summary: .cache/load/aggregator-isolation-2026-10-09T15-37-25-357Z.json

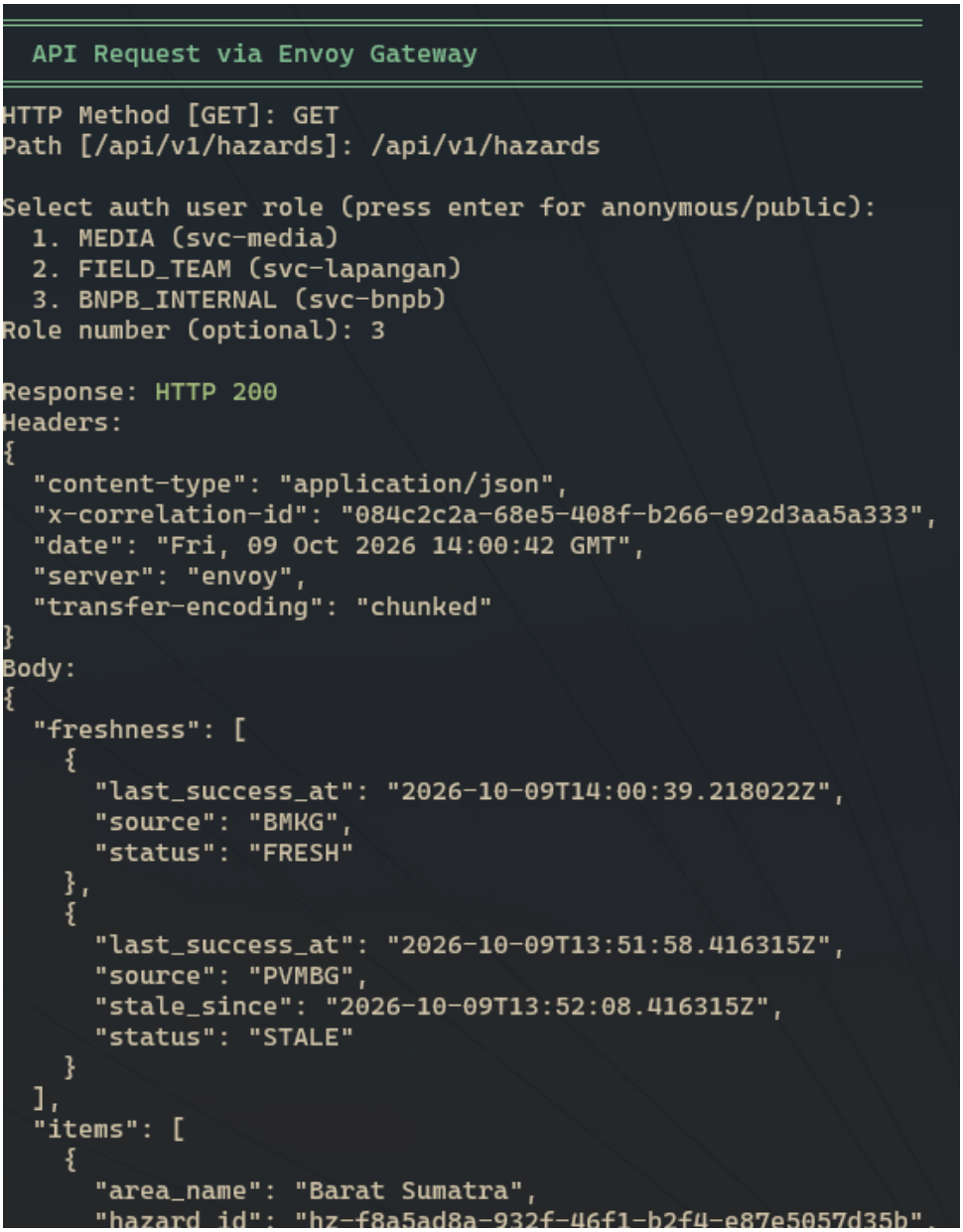
running (1m43.9s), 00/20 VUs, 29099 complete and 40 interrupted iterations
warmup           ✓ [======================================] 20 VUs  5s
baseline         ✓ [======================================] 10 VUs  30s
baseline_filler  ✓ [======================================] 10 VUs  30s
contention_bmkg  ✓ [======================================] 10 VUs  1m0s
contention_pvmbg ✓ [======================================] 10 VUs  1m0s
```

Gambar 3.2: Hasil isolasi: probe PVMBG sekitar 2,92 detik dan p95 BMKG 87,09 ms saat kontensi, di bawah target 300 ms (sumber: `P2/1.png`).

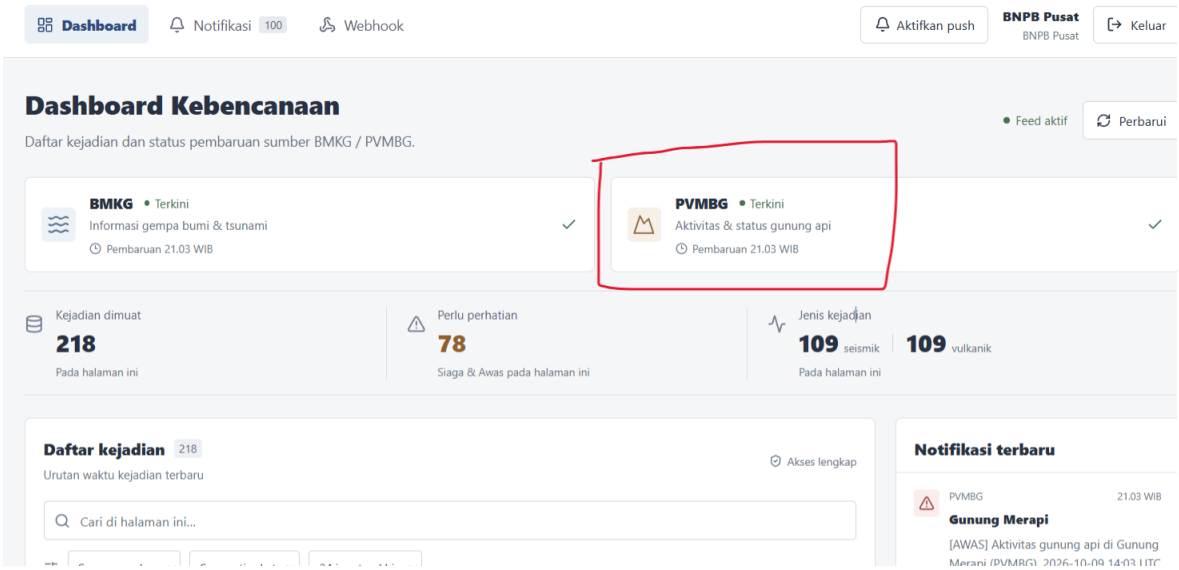
3.4.3 Outage, Data Stale, dan Pemulihan

Urutan screenshot demo outage terdapat pada `evidence/P2/3-1.png` sampai `evidence/P2/3-5.png`. Outage PVMBG diaktifkan melalui kontrol admin, kemudian dashboard memperlihatkan BMKG tetap terkini dan PVMBG terlambat. Respons `GET/api/v1/hazards` melalui Envoy tetap HTTP 200 dengan metadata BMKG `FRESH` dan PVMBG `STALE`. Respons juga menampilkan `last_success_at` dan `stale_since` untuk PVMBG, sehingga gangguan sumber tidak ditampilkan sebagai tidak adanya kejadian. Setelah outage dinonaktifkan, dashboard kembali memperlihatkan kedua sumber terkini.

Ringkasan pengujian otomatis, `evidence/problem_set_report.md`, mencatat tiga tes P2 berhasil dalam 74,97 detik. Tes outage pada `tests/test_problem_2.py` memeriksa data PVMBG tersimpan tetap tersedia, HTTP 200, status stale, serta pemulihan ke fresh setelah



Gambar 3.3: Respons API HTTP 200 saat outage PVMBG: BMKG tetap fresh, sedangkan PVMBG ditandai stale beserta waktu kesegarannya (sumber: `P2/3-3.png`).



Gambar 3.4: Dashboard sesudah outage dinonaktifkan memperlihatkan BMKG dan PVMBG kembali terkini (sumber: `P2/3-5.png`).

outage dinonaktifkan; prosedur tes tersebut tidak menjalankan restart. Namun, gambar manual tidak melampirkan identitas dan waktu mulai container sebelum/sesudah, sehingga gambar saja tidak membuktikan tidak terjadi restart di luar prosedur.

Narasi kriteria pada ringkasan pengujian otomatis merupakan template ringkasan, bukan rekaman angka latensi atau konfigurasi efektif. Pengukuran 50 VU/60 detik dan validasi delay PVMBG karena itu dirujukkan ke JSON k6 beban dan isolasi. Bukti ini mendukung skenario yang terekam, tanpa mengklaim ketahanan outage berkepanjangan atau availability di luar jendela pengujian.

Bab 4

P3: Autentikasi, Otorisasi, dan Trust

4.1 Rancangan Solusi

4.1.1 Pemisahan Domain Kepercayaan

BMKG menerima API key melalui `X-BMKG-Key`; PVMBG menerima token opaque melalui `Authorization:Bearer`. Keduanya memvalidasi secret milik sendiri. Endpoint data dan kontrol PVMBG terlindungi, sedangkan health dapat dipakai tanpa kredensial. Aggregator mengirim header yang sesuai pada adapter sumber; kredensial pengguna BNPB tidak diteruskan ke mock.

Helper `tools/setup_mock_env.py` membuat dua nilai acak berbeda dengan generator secret, menolak penggunaan nilai sama, dan mempertahankan secret yang sudah valid. Helper hanya memprovisikan konfigurasi; layanan dapat dijalankan tanpa helper bila environment sudah diisi. SQLite menyimpan data mock, bukan menjadi sumber API key. Seorang anggota tim yang menyiapkan environment sendiri memperoleh secret lokal sendiri dan harus menyamakan konfigurasi pemanggil serta penerima pada instalasinya.

Untuk pengguna, Zitadel menjadi identity provider sekaligus authorization server. OIDC memberikan identitas login melalui ID token; OAuth menyediakan access token untuk akses API [3]. Akun `media-demo`, `lapangan-demo`, dan `bnpb-demo` mempunyai subject dan kredensial terpisah di PostgreSQL Zitadel. Bootstrap mengatur role, grant, aplikasi OIDC, serta TTL access token default 60 detik. Akun service untuk tes API juga diprovisikan terpisah. SQLite Auth bridge hanya menyimpan sesi dan transaksi login.

4.1.2 Login, Cookie, dan Pemanggilan API

Alur login menggunakan Authorization Code dengan PKCE S256. PKCE mengikat pertukaran authorization code dengan verifier transaksi, sehingga kepemilikan code saja tidak cukup untuk menukarnya [4].

1. Browser membuka `/auth/login`. Bridge membuat state, nonce, verifier PKCE, dan cookie transaksi, lalu mengarahkan pengguna ke hosted Login UI Zitadel. Password dikirim ke provider.
2. Callback memvalidasi transaksi browser dan menukar authorization code. Library `open id-client` memeriksa ID token, issuer, audience, nonce, serta masa berlaku. Introspeksi access token harus menghasilkan identitas aktif dengan audience project dan role/scope yang dikenal.
3. Bridge menyimpan refresh token serta payload sesi dalam SQLite yang dienkripsi AES-256-GCM. Browser memperoleh cookie `bnpb_access` berisi access token opaque asli dan `bnpb_s ession` berisi ID sesi acak.

4. Kedua cookie bersifat HttpOnly, SameSite=Lax, host-only, dan Path=/; Secure diaktifkan pada HTTPS. Respons `/auth/session` hanya mengembalikan profil, scope, expiry, dan CSRF token. Token akses/refresh tidak dimasukkan ke JSON ataupun penyimpanan JavaScript.
5. Untuk `/api/*`, filter Lua Envoy memindahkan access cookie menjadi header bearer dan menghapus cookie sebelum meneruskan request. Campuran bearer eksplisit dan cookie Auth ditolak agar sumber kredensial tidak ambigu.
6. Envoy memanggil sidecar `token-exchange` melalui filter `ext_authz`. Sidecar tersebut menggunakan confidential client ke Zitadel dan meminta JWT melalui mekanisme RFC 8693 [5]. JWT diteruskan melalui API Gateway sebagai metadata otorisasi gRPC. Service bisnis memverifikasi token dan memeriksa izin method.

Target Rust `zitadel-claims` menerima hook Actions v2 sebelum pembentukan access token dan userinfo/introspeksi. Target memeriksa signature HMAC-SHA256 beserta timestamp panggilan dari Zitadel, lalu membentuk `role` dan `scopes` dari grant pada project/organisasi yang diizinkan. Claim tidak ditentukan oleh input role di frontend. Target ini tidak dibuka sebagai endpoint publik pada Envoy.

4.1.3 Hak Akses dan Pembatasan Data

Hak akses	MEDIA	FIELD_TEAM	BNPB_INTERNAL
Hazard ringkasan	Diizinkan	Diizinkan	Diizinkan
Hazard penuh/mentah	Ditolak	Diizinkan	Diizinkan
Baca notifikasi	Diizinkan	Diizinkan	Diizinkan
Baca webhook	Diizinkan sesuai kepemilikan	Diizinkan sesuai kepemilikan	Diizinkan sesuai kepemilikan
Buat/ubah webhook	Ditolak	Diizinkan	Diizinkan

Tabel 4.1: Scope role yang dibentuk oleh target Actions. Pemeriksaan izin berada pada backend.

Untuk hazard, Aggregator menolak permintaan `VIEW_FULL` tanpa `hazards:read:full`. Proyeksi summary hanya memuat tujuh field ringkasan Bab 2; koordinat, referensi asli, dan attributes tidak disertakan. API Gateway menerjemahkan representasi Protobuf menjadi bentuk OpenAPI dan menolak bentuk backend yang melanggar proyeksi. Menyembunyikan tombol detail pada UI hanya mendukung pengalaman pengguna; pengamanan utama adalah pengecekan scope dan pemilihan proyeksi pada server.

4.1.4 Refresh, Logout, dan CSRF

SPA menjadwalkan refresh sekitar lima detik sebelum expiry serta dapat memulihkan sesi setelah 401. Bridge menggunakan refresh token server dan menggabungkan request refresh bersamaan untuk sesi yang sama. Rotasi data sesi dan pencatatan hash token yang ditolak dilakukan dalam transaksi SQLite. Pengguna tidak perlu memasukkan password lagi selama refresh token dan sesi masih berlaku. Kode bridge juga mencoba mencabut access token lama pada Zitadel setelah rotasi.

POST refresh/logout mensyaratkan Origin aplikasi dan kecocokan nilai `X-CSRF-Token` dengan sesi. Pada mutasi API berbasis cookie, Envoy memeriksa Origin dan keberadaan header CSRF; Envoy tidak melakukan perbandingan dengan SQLite bridge. Kebijakan ini bergantung pada

origin yang tepat dan tidak dibukanya CORS untuk origin sembarang. Bearer eksplisit bagi klien API mengikuti jalur tersendiri tanpa cookie browser.

Logout menghapus sesi lokal, membersihkan cookie, dan mencoba revokasi provider. Browser diarahkan ke logout Zitadel untuk mengakhiri SSO. Login UI V2 dapat menampilkan tindakan *End Session*; penghapusan cookie aplikasi saja tidak selalu mengakhiri sesi provider. Implementasi ada di `services/auth-bridge/src/server.mjs`, `provider.mjs`, dan `store.mjs`.

4.1.5 Analisis: Mengapa Mekanisme Ini Tidak Membuka Celah Baru

Mekanisme Auth menambah komponen dan endpoint baru: Auth Bridge (`/auth/*`), cookie sesi, filter Lua pada Envoy, Token Exchange Sidecar, dan hook `zitadel-claims`. Setiap tambahan tersebut merupakan permukaan serangan baru, sehingga rancangan memeriksa celah yang dapat muncul dari masing-masing komponen dan memastikan setiap celah tersebut ditutup oleh kontrol tertentu. Tabel 4.2 merangkum analisis ini.

Komponen baru	**Celah yang mungkin muncul**	**Kontrol yang menutupnya**
Token di browser	Pencurian token melalui XSS atau ekstensi yang membaca `localStorage`.	Access token hanya berada pada cookie HttpOnly; refresh token tidak pernah dikirim ke browser. `/auth/session` dan `/auth/refresh` hanya mengembalikan profil, scope, expiry, dan CSRF token.
Cookie sebagai kredensial	CSRF: situs lain memicu request ber-cookie atas nama pengguna.	SameSite=Lax, cookie host-only, pemeriksaan Origin, dan header `X-CSRF-Token` yang dicocokkan dengan sesi untuk refresh/logout; mutasi API berbasis cookie wajib membawa Origin dan header CSRF.
Alur login `/auth/login` dan callback	Login CSRF, injeksi authorization code, replay ID token, open redirect.	`state` dan `nonce` acak; PKCE S256; transaksi login terikat cookie browser dan kedaluwarsa dalam 10 menit. Tujuan redirect setelah login adalah `appUrl` dari konfigurasi, bukan parameter request.
Penyimpanan sesi Auth Bridge	Kebocoran refresh token dari file SQLite; pemakaian ulang refresh token lama.	Payload sesi dienkripsi AES-256-GCM dan ID sesi disimpan sebagai hash SHA-256. Refresh token dirotasi; hash token lama dicatat sebagai ditolak.
Filter Lua cookie → bearer	Kebingungan sumber kredensial; cookie bocor ke layanan internal.	Request yang membawa bearer eksplisit sekaligus cookie Auth ditolak. Cookie dihapus sebelum request diteruskan ke upstream.
Token Exchange Sidecar	Sidecar disalahgunakan sebagai penerbit JWT bagi token sembarang.	Sidecar tidak dipublikasikan ke host dan hanya dipanggil Envoy melalui `ext_authz`. Pertukaran memakai confidential client dan hanya berhasil untuk access token aktif yang diterbitkan Zitadel; token kedaluwarsa ditolak (Gambar 4.6).
Hook `zitadel-claims`	Pemalsuan role/scope untuk eskalasi hak akses.	Tanda tangan HMAC-SHA256 dan timestamp setiap panggilan diverifikasi; hook tidak dirutekan oleh Envoy. Role dibentuk dari grant Zitadel, bukan dari input frontend.
Port gRPC layanan	Bypass gateway dengan memanggil layanan secara langsung.	Setiap layanan backend memverifikasi JWT (issuer, expiry, algoritme, kunci JWKS) dan memeriksa scope per method secara mandiri, sehingga melewati gateway tidak melewati otorisasi.
Kredensial mock	Kredensial pengguna BNPB atau milik satu instansi dipakai ke instansi lain.	Secret BMKG dan PVMBG berbeda dan disimpan di environment; kredensial pengguna tidak diteruskan ke mock (Gambar 4.1).

Tabel 4.2: Analisis celah yang mungkin muncul dari komponen Auth baru dan kontrol yang menutupnya.

Selain kontrol per komponen, dua prinsip membatasi dampak bila satu kontrol gagal. Pertama, TTL access token 60 detik membatasi masa pakai token yang bocor. Kedua, keputusan otorisasi

dibuat di backend berdasarkan scope pada JWT, sehingga celah pada UI, misalnya tombol detail yang terlihat, tidak memberikan akses data. Risiko yang masih tersisa, yaitu revokasi asinkron, audience yang dikosongkan pada Compose lokal, transport plaintext, dan akun broker bersama, bukan celah yang tidak disadari. Risiko tersebut merupakan tradeoff lingkungan lokal yang dicantumkan secara eksplisit pada Batasan Solusi beserta konfigurasi yang diperlukan untuk menutupnya di luar lingkungan lokal.

4.2 Justifikasi dan Alternatif

API key/opaque token sederhana dipakai untuk mock agar perbedaan protokol autentikasi antarsumber tetap terlihat tanpa membangun provider untuk setiap instansi. Pemisahan secret membatasi penggunaan kredensial lintas instansi. Konsekuensinya, rotasi dan distribusi secret mock tetap menjadi tugas konfigurasi.

OIDC terpusat menghindari pembuatan form password, hash password, dan penerbitan JWT sendiri di frontend BNPB. Dibanding menyimpan bearer token dalam localStorage, cookie HttpOnly mengurangi keterbacaan token oleh JavaScript, tetapi menambah kebutuhan perlindungan CSRF. Bridge dipakai karena refresh token serta client secret perlu berada di server. Dibanding introspeksi pada setiap service, token exchange di edge memberikan format JWT untuk validasi backend. Biayanya adalah dependensi pada provider pada jalur masuk dan tambahan konfigurasi client.

Alternatif mTLS dapat mengautentikasi identitas workload antarlayanan dengan lebih kuat, tetapi memerlukan pengelolaan sertifikat. Implementasi M1 menggunakan JWT pengguna serta token internal `x-service-token` pada jalur yang mendukungnya. Token internal bukan identitas pengguna dan tidak boleh diberikan kepada browser. Pembagian broker user per consumer akan memberi isolasi lebih baik daripada satu akun broker bersama, tetapi belum diterapkan seragam pada Compose.

4.3 Batasan Solusi

- Revokasi access token lama pada provider dilakukan asinkron dan kegagalannya ditoleransi. Daftar penolakan lokal bridge tidak dibaca Envoy maupun sidecar token exchange. Karena itu penolakan seketika pada gateway tidak dapat dijamin jika revokasi gagal; token lama mungkin berlaku hingga expiry. JWT hasil exchange yang sudah diterbitkan juga memiliki siklus validasinya sendiri.
- Sebagai tradeoff pengembangan lokal, Compose mengosongkan `AUTH_AUDIENCE` pada Aggregator dan Webhook Provider sehingga klaim audience tidak diperiksa. Dengan begitu nilai audience tidak perlu disesuaikan dengan hasil bootstrap Zitadel setiap kali stack lokal dibuat ulang. Kedua verifier memeriksa audience apabila variabel tersebut diisi, sehingga deployment di luar lokal perlu mengisinya.
- Compose lokal memakai HTTP dan gRPC plaintext, bukan mTLS. Koordinasi refresh dan SQLite sesi ditujukan untuk satu instance bridge; replikasi membutuhkan koordinasi/penyimpanan bersama.
- Secret mock, Auth, PAT, password canonical database, dan file Firebase dikeluarkan dari Git dan diberikan per variabel kepada service yang memerlukannya. Sebagai tradeoff pengembangan lokal, Compose hanya memprovisikan satu akun RabbitMQ (`RABBITMQ_DEFAULT_*`) yang dipakai bersama oleh producer dan seluruh consumer, sehingga stack lokal cukup dijalankan dengan satu konfigurasi broker. Deployment di luar lokal memerlukan akun broker per layanan dengan izin exchange/queue minimum agar distribusi secret minimum terpenuhi.
- Provider yang tidak tersedia dapat menyebabkan token exchange gagal meskipun data hazard masih tersimpan. Graceful degradation terhadap mock pada P2 tidak otomatis menghi-

langkan dependensi Auth ini.

4.4 Proof-of-Concept dan Keterlacakan

Bukti P3 menggabungkan hasil uji kredensial mock dalam `mock-container-2026-09-30.json`, pemeriksaan sesi browser historis dalam `auth-browser-2026-10-06.json`, tes unit validasi JWT backend dalam `aggregator-unit-2026-10-09.jsonl`, laporan pengujian stack dalam `problem_set_report.md`, dan tangkapan layar demonstrasi P3. Laporan pengujian stack mencatat empat tes P3 lulus tanpa error/failure maupun skip, dengan durasi 61,37 detik. Angka tersebut adalah hasil suite yang disimpan; kesimpulan per alur di bawah mengikuti request dan respons yang benar-benar terlihat pada artefak.

4.4.1 Isolasi Kredensial Sumber

Hasil uji kredensial mock merekam penolakan tujuh request tidak sah pada kedua mock. Pada laporan pengujian stack, `test_01_cross_service_credentials_rejected` menguji kredensial silang dan request tanpa kredensial pada endpoint data `/seismic-events` serta `/volcanic-reports`. Keduanya merupakan dasar pengujian isolasi kredensial.

Artefak terbaru `P3/1-new.png` pada commit `4a1973b` memperlihatkan dua penolakan HTTP 401: bearer PVMBG ditolak BMKG, dan API key BMKG ditolak PVMBG. Respons keduanya menyatakan kredensial instansi yang sesuai diperlukan. Bukti ini menggantikan `P3/1.png` sebagai observasi manual utama dan mendukung isolasi kredensial dua arah. Artefak lama tetap dipertahankan sebagai riwayat.

Kontrol dengan kredensial benar pada gambar terbaru masih menghasilkan 404 untuk BMKG dan 405 untuk PVMBG akibat pemilihan route/method. Karena itu label *Authorized* pada keluaran menu tidak ditafsirkan sebagai keberhasilan membaca data. Keberhasilan akses dengan kredensial benar pada endpoint data tetap merujuk hasil uji kredensial mock dan laporan pengujian stack; gambar terbaru digunakan untuk dua penolakan kredensial silang yang terlihat.

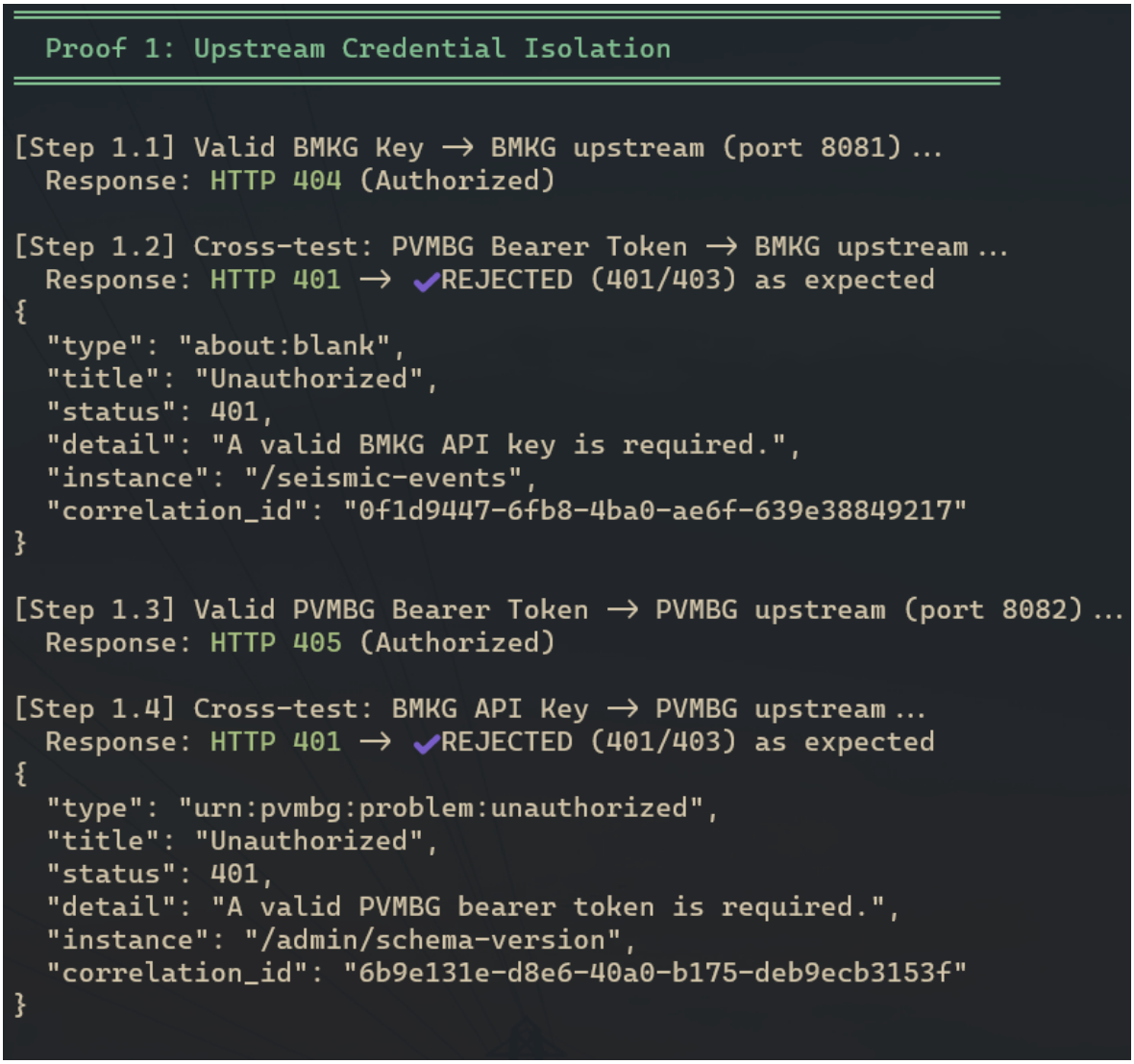
4.4.2 Otorisasi Ringkasan dan Data Penuh

Tangkapan layar demonstrasi P3 memperlihatkan request akun service ber-role MEDIA ke `/api/v1/hazards` memperoleh 200 dan hanya tujuh field ringkasan. Permintaan eksplisit `view=full` dengan role yang sama memperoleh 403 dari server, beserta correlation ID. Ketika role FIELD_TEAM meminta proyeksi penuh, respons 200 memuat `source_ref_id`, koordinat, dan attributes. Hal ini menunjukkan bahwa pembatasan data berlaku pada API, bukan hanya pada tampilan.

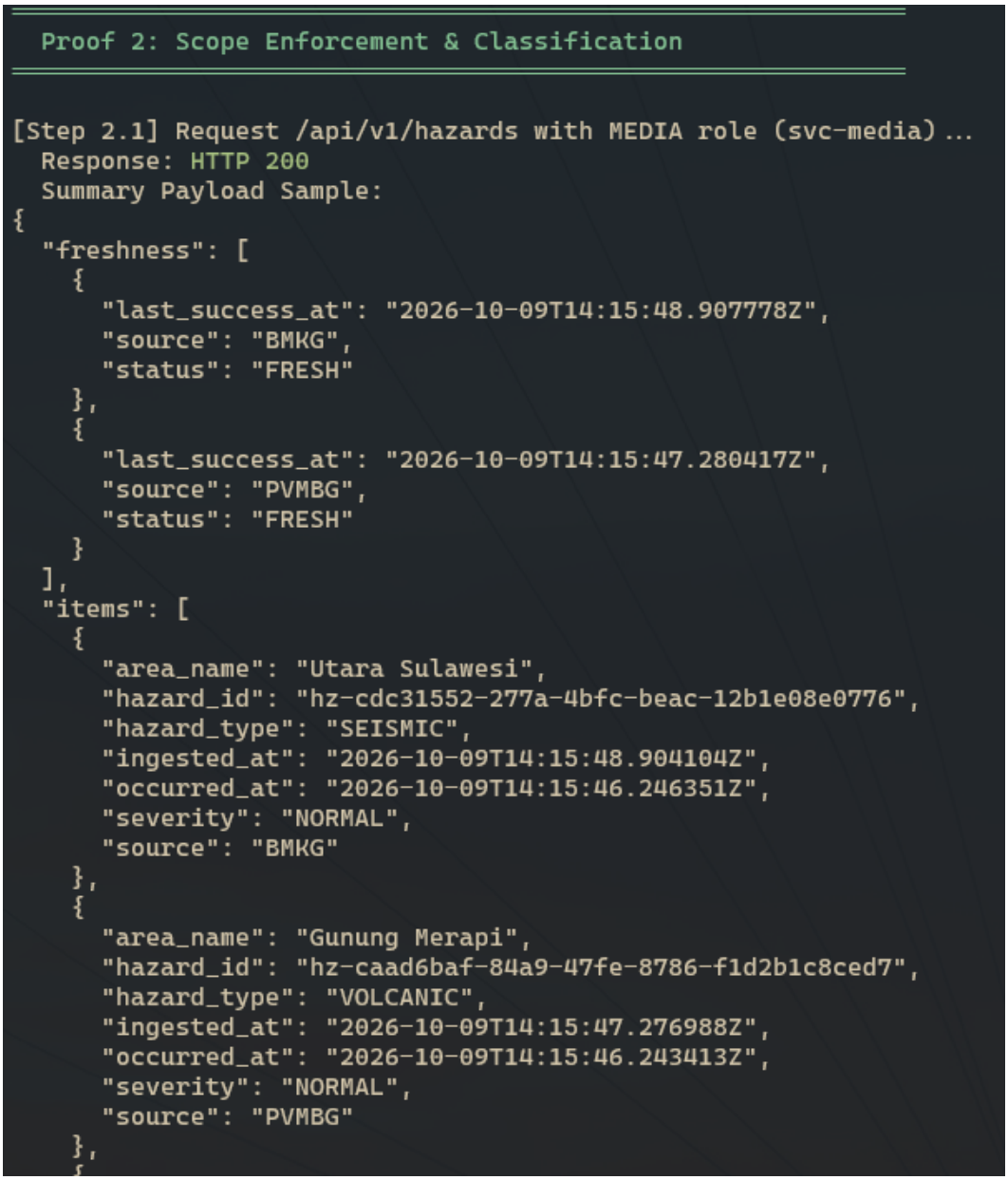
Laporan pengujian stack melengkapi tangkapan layar dengan pemeriksaan proyeksi penuh untuk FIELD_TEAM dan BNPB_INTERNAL serta penolakan detail hazard penuh bagi MEDIA. Identitas pada tes tersebut adalah akun service `svc-media`, `svc-lapangan`, dan akun service BNPB. Demonstrasi ini membuktikan enforcement role pada request API; login tiga akun pengguna melalui hosted UI tercatat terpisah pada pemeriksaan sesi browser 6 Oktober 2026.

4.4.3 Expiry Token dan Pembaruan Akses

Pada Gambar 4.5 dan Gambar 4.6, token akun service Tim Lapangan mula-mula menghasilkan 200. Setelah menunggu 62 detik, token yang sama menghasilkan 401 dengan detail `tokenexchangerejected:STSreturned400`; penolakan terjadi di edge sebelum request mencapai layanan bisnis. Penerbitan token baru menghasilkan akses 200, sedangkan token pertama tetap ditolak dengan 401. Dengan demikian, artefak manual mendukung berlakunya expiry pada jalur



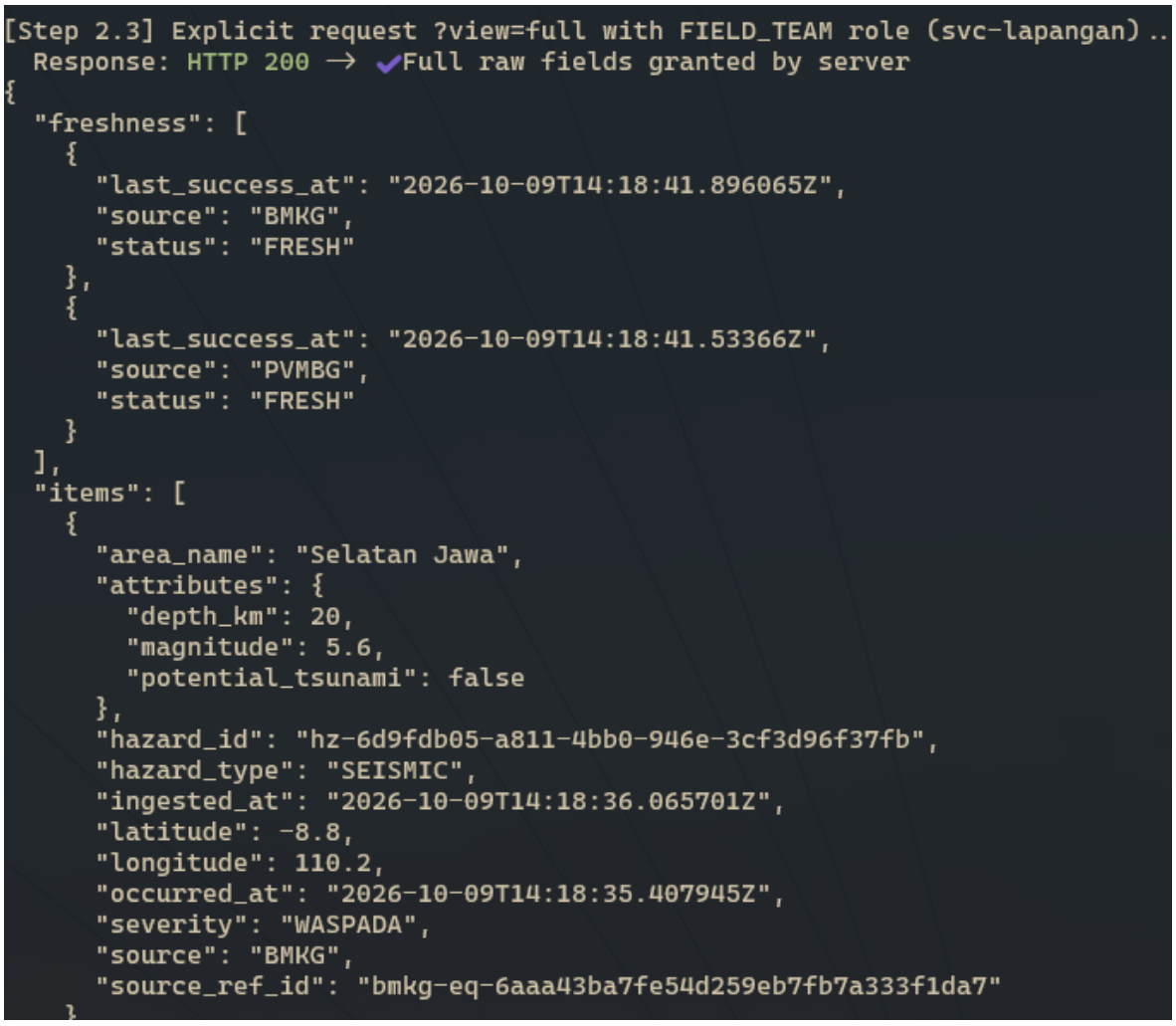
Gambar 4.1: Kredensial silang ditolak dengan HTTP 401 pada BMKG maupun PVMBG. Status 404/405 pada kontrol terpisah tidak digunakan sebagai bukti keberhasilan akses data. Sumber: `P3/1-new.png`.



Gambar 4.2: Respons summary untuk role MEDIA; record tidak memuat koordinat, referensi sumber, ataupun attributes. Sumber: `P3/2-1.png`.

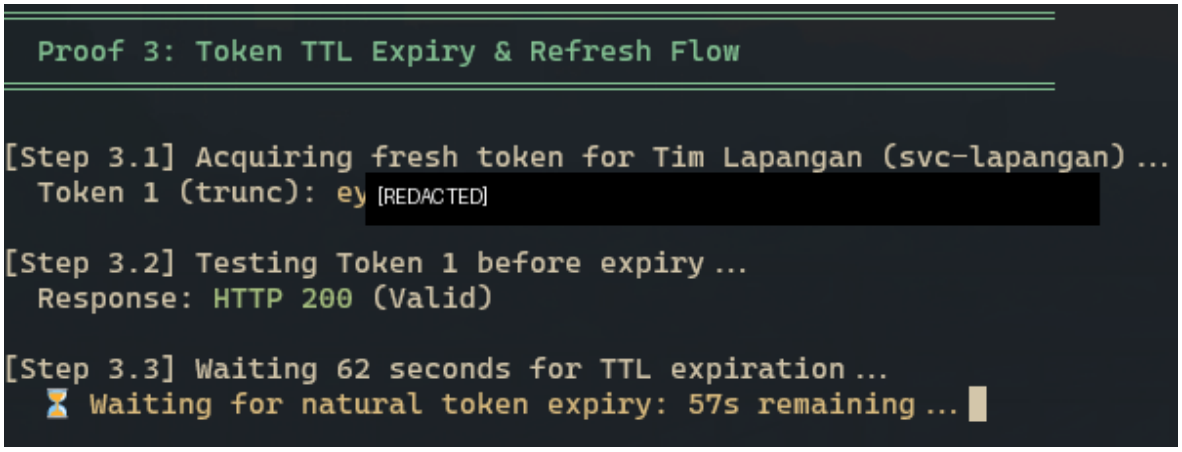


Gambar 4.3: Permintaan eksplisit data penuh oleh MEDIA ditolak server dengan HTTP 403. Sumber: `P3/2-2.png`.

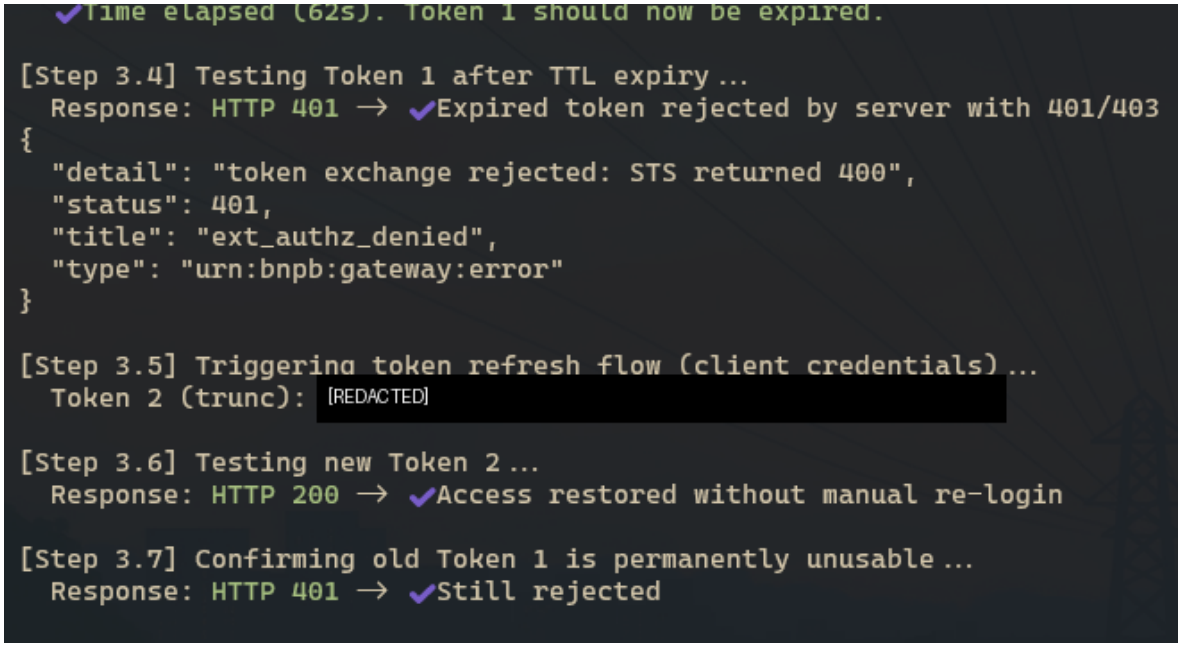


Gambar 4.4: Role FIELD_TEAM memperoleh HTTP 200 dan field penuh. Sumber: `P3/2-3.png`.

gateway serta pemulihan akses menggunakan token baru. Fragmen token pada kedua gambar ditutup sebelum dimuat ke laporan; gambar asli tetap tersimpan sebagai artefak rujukan.



Gambar 4.5: Token pertama akun service Tim Lapangan diterima (HTTP 200), lalu pengujian menunggu expiry alami 62 detik. Fragmen token ditutup. Sumber: `P3/3-1.png`.



Gambar 4.6: Setelah TTL berakhir, token pertama ditolak HTTP 401 oleh token exchange; token pengganti diterima HTTP 200 dan token pertama tetap ditolak. Fragmen token ditutup. Sumber: `P3/3-2.png`.

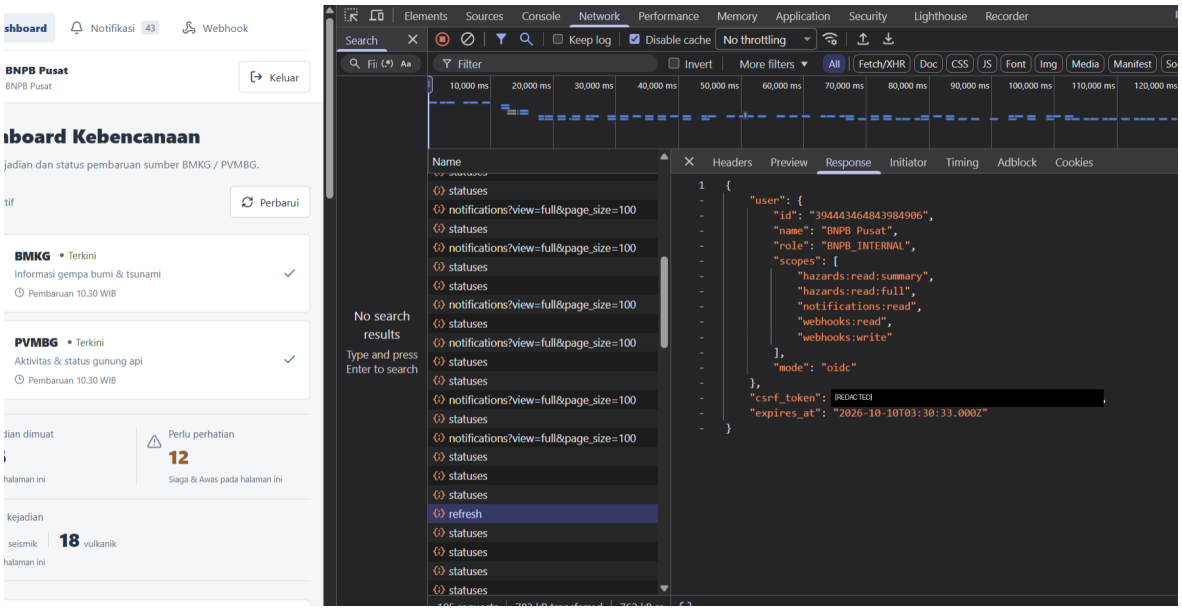
Menu demo dan `test_03_token_ttl_expiry_and_refresh_flow` meminta token pengganti dengan grant `client_credentials`. Ini adalah penerbitan ulang token service; tidak melibatkan refresh token sesi pengguna. Penolakan token pertama setelah expiry juga tidak mengukur kecepatan revokasi token yang masih berlaku.

Tangkapan layar tambahan pada Gambar 4.7 dan Gambar 4.8 merekam respons request `refresh` pada browser dengan dashboard aktif. Kedua respons memuat identitas pengguna yang sama, nama BNPB Pusat, role `BNPB_INTERNAL`, serta mode `oidc`. Nilai `expires_at` berubah dari 03:30:33 menjadi 03:31:28 UTC pada 10 Oktober 2026, atau bertambah 55 detik, sementara request polling dashboard berlanjut. Ini mendukung pembaruan sesi browser BNPB Pusat pada stack terbaru selama penggunaan dashboard, sesuai penjadwalan refresh sekitar lima detik

sebelum TTL 60 detik berakhir.

Field aman	Respons refresh pertama	Respons refresh berikutnya
Nama pengguna	BNPB Pusat	BNPB Pusat
Role dan mode	`BNPB_INTERNAL`, `oidc`	`BNPB_INTERNAL`, `oidc`
`expires_at` (UTC)	`2026-10-10T03:30:33.000Z`	`2026-10-10T03:31:28.000Z`

Tabel 4.3: Ringkasan field dari screenshot `P3/3-1-alternative.png` dan `P3/3-2-alternative.png`. Identitas tetap sama dan expiry bertambah 55 detik; nilai CSRF tidak disalin.

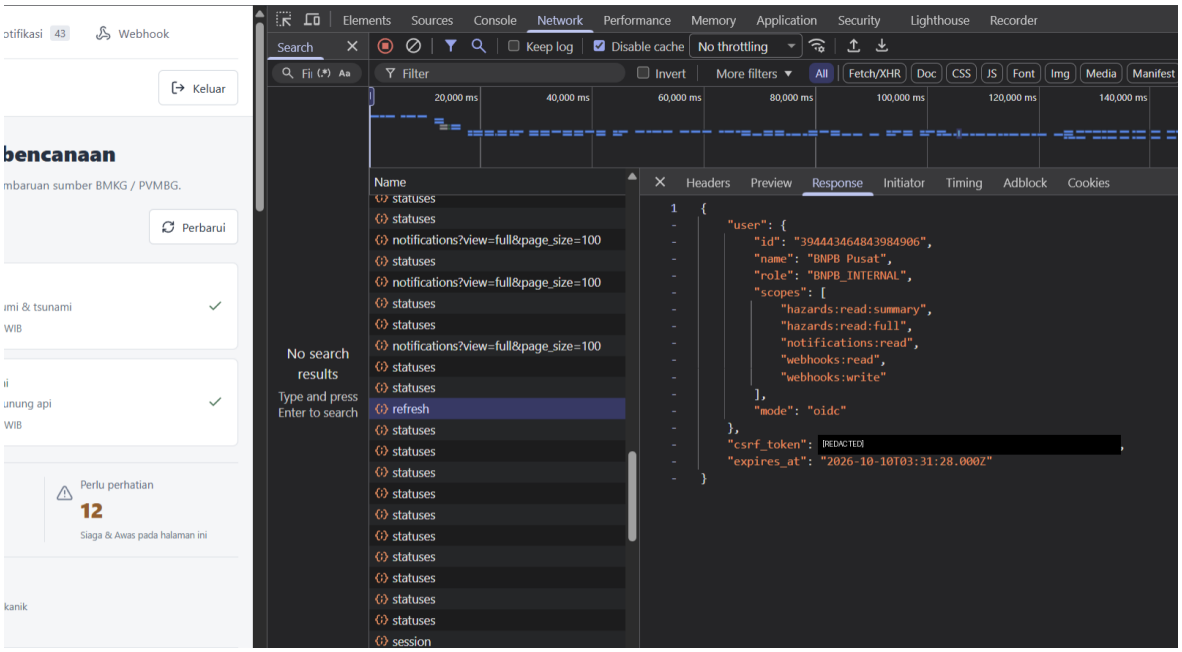


Gambar 4.7: Respons `refresh` pertama pada dashboard BNPB Pusat: profil, role `BNPB_INTERNAL`, mode `oidc`, dan `expires_at` 03:30:33 UTC. Body tidak memuat access/refresh token; nilai CSRF ditutup. Sumber: `P3/3-1-alternative.png`.

Kedua screenshot tersebut melengkapi hasil uji sesi browser historis, yang mencatat 45 pemeriksaan browser/provider pada 6 Oktober dengan arsitektur bridge saat itu. Cakupan screenshot refresh terbaru adalah pembaruan sesi BNPB Pusat; bukti tersebut tidak memperlihatkan sesi pengguna Tim Lapangan dibiarkan kedaluwarsa lalu dipulihkan, ataupun request eksplisit memakai token lama setelah refresh. Kesimpulan kedua skenario itu tetap dibedakan dari refresh browser yang sudah terlihat.

4.4.4 Konfigurasi Secret dan Ringkasan Hasil

Tangkapan layar `P3/4-1.png` dan `P3/4-2.png` memperlihatkan penelusuran pola key/token/secret pada repository dan tersedianya placeholder konfigurasi. Pengujian stack memeriksa variabel yang diwajibkan oleh suite pada `.env.example` serta pola tertentu untuk kredensial literal. Pemeriksaan ini mendukung penggunaan environment pada komponen terkait, dengan cakupan pola terbatas: kredensial pendek atau bentuk YAML tanpa tanda kutip tidak seluruhnya tercakup. Sesudah tangkapan layar tersebut diambil, password canonical database dipindahkan dari nilai literal pada Compose ke variabel environment `AGGREGATOR_DB_PASSWORD` yang diisi secara lokal; helper setup tidak membangkitkan nilai ini. Akun RabbitMQ bersama tetap merupakan tradeoff pengembangan lokal yang telah dijelaskan sebelumnya.



Gambar 4.8: Respons `refresh` berikutnya pada sesi yang sama: identitas tidak berubah dan `expires_at` maju ke 03:31:28 UTC tanpa login ulang. Nilai CSRF ditutup. Sumber: `P3/3-2-alternative.png`.

Kriteria	**Bukti tersedia**	**Kesimpulan dan batas cakupan**
Isolasi kredensial mock	Hasil uji kredensial mock; laporan stack; `P3/1-new.png`.	Kredensial silang ditolak 401 pada kedua mock. Kontrol akses data yang berhasil mengacu hasil uji kredensial mock dan laporan stack.
Pembatasan role API	Tes scope pada laporan stack; `P3/2-1.png` sampai `P3/2-3.png`.	MEDIA summary 200/full 403; FIELD_TEAM full 200. Pengujian stack juga memeriksa BNPB_INTERNAL.
Expiry dan token pengganti	Laporan stack; `P3/3-1.png` dan `P3/3-2.png`.	Token service lama 401 sesudah 62 detik; token baru 200. Alur memakai client credentials.
Login, cookie, refresh browser	Pemeriksaan sesi browser 6 Oktober; `P3/3-1-alternative.png` dan `P3/3-2-alternative.png`.	Sesi browser BNPB Pusat tetap aktif, identitas sama, dan expiry bertambah 55 detik. Expiry sesi Tim Lapangan dan penolakan token lama sesudah refresh tidak ditunjukkan oleh screenshot baru.
Validasi JWT backend	Hasil tes unit Aggregator: 15 tes utama Auth lulus.	Memeriksa issuer, expiry, algoritme, rotasi JWKS, service token, dan provider unavailable dengan server tes.
Konfigurasi tanpa secret literal	Laporan stack; `P3/4-1.png` dan `P3/4-2.png`.	Pemeriksaan pola dan placeholder tersedia; password canonical database berasal dari environment. Akun RabbitMQ bersama merupakan tradeoff pengembangan lokal.

Tabel 4.4: Hasil P3 dengan pembedaan alur service, sesi browser, dan konfigurasi.

Nilai token, cookie, password, dan client secret tidak diperlukan dalam laporan. Gambar expiry yang menyertakan fragmen token dan dua screenshot refresh yang memuat CSRF token dimuat dalam versi yang bagian sensitifnya telah ditutup (`*-redacted.png`); artefak asli tetap disimpan tanpa perubahan. Panduan pengujian sesi browser serta E2E tersedia di `docs/auth-testing.md` dan `docs/e2e-testing.md`.

Bab 5

P4: Layanan dan Penyimpanan

5.1 Rancangan Solusi

5.1.1 Pembagian Domain dan Batas Layanan

Batas layanan ditarik menurut domain bisnis, bukan menurut lapisan teknis. Pertanyaan yang dipakai untuk menentukan batas adalah: kemampuan bisnis apa yang dilayani, data apa yang menjadi miliknya, dan kepada siapa ia bergantung ketika berjalan. Hasilnya adalah satu domain inti yang menghasilkan data hazard kanonis dan tiga domain penyampaian yang masing-masing mengonsumsi data itu dengan semantik berbeda. Identitas dan kontrak publik ditempatkan sebagai domain pendukung di tepi sistem.

Tabel 5.1: Pembagian domain dan batas layanan

Domain	**Layanan**	**Kemampuan bisnis**	**Alasan dipisah**
Integrasi sumber (inti)	Aggregator (Go)	Mengambil data BMKG/PVMBG, memetakan ke `HazardEvent`, menjadi satu-satunya penulis data kanonis, dan memublikasikan perubahan.	Satu-satunya domain yang memahami format dan kredensial instansi sumber; aturan pemetaan berubah mengikuti sumber, bukan mengikuti klien.
Distribusi real-time	Event Live Feed (Rust)	Mengalirkan event hazard ke klien yang sedang terhubung, dengan filter, proyeksi, dan replay jangka pendek.	Beban berupa banyak koneksi berumur panjang; semantiknya *best-effort* dalam jendela replay, berbeda dari pengiriman yang harus tercatat.
Integrasi pihak ketiga	Webhook Provider (Go)	Mengelola langganan webhook dan mengirim callback bertanda tangan ke sistem eksternal.	Bergantung pada endpoint di luar kendali BNPB yang bisa lambat atau mati; retry dan log pengiriman tidak boleh menahan domain lain.
Peringatan bahaya	Danger Notification (C#)	Memilih hazard SIAGA/AWAS, menyimpan notifikasi, dan mengirim push melalui FCM.	Memiliki aturan bisnis sendiri (ambang severity, revisi notifikasi) dan dependensi eksternal FCM.
Identitas dan akses (pendukung)	Zitadel, Auth Bridge, sidecar	Login, sesi, penerbitan token, dan pembentukan role/scope.	Menjadi dependensi lintas domain; dipisahkan agar domain bisnis cukup memverifikasi JWT tanpa mengelola kredensial pengguna.
Kontrak publik (pendukung)	Envoy, API Gateway (Go)	Routing, validasi OpenAPI, dan penerjemahan HTTP/SSE ke gRPC.	Bukan pemilik data; memisahkan bentuk API publik dari kontrak internal tiap domain.

Tiga domain penyampaian sengaja tidak digabung walaupun ketiganya mengonsumsi event yang sama. Ketiganya berbeda pada tiga hal: tujuan penyampaian (klien yang sedang terhubung, sistem eksternal, perangkat pengguna), jaminan yang diberikan (replay terbatas, pengiriman tercatat dengan retry, push satu kali per revisi), dan dependensi eksternal (tidak ada, endpoint callback, FCM). Bila digabung, endpoint webhook yang lambat atau FCM yang gagal dapat menahan aliran feed. Dengan pemisahan, kegagalan satu domain hanya menumpuk backlog pada queue miliknya (Bab 6).

Hubungan antardomain hanya melalui dua jenis kontrak: event `HazardEventPublished` dari domain inti ke domain penyampaian, dan gRPC untuk query, misalnya Danger Notification yang menanyakan status kesegaran sumber ke Aggregator. Kontrak Protobuf menjadi batas lintas bahasa, sehingga setiap domain dapat memakai bahasa yang paling sesuai: Go untuk Aggregator dan Webhook, Rust untuk stream asinkron Feed, dan C# untuk Notification. Compose membangun image dari direktori masing-masing layanan, sehingga satu layanan dapat dihentikan, dibangun ulang, dan dijalankan kembali tanpa menyentuh layanan lain.

Setiap domain memiliki datanya sendiri (*database per service* [10]): Aggregator memiliki Canonical Store, Webhook Provider memiliki langganan dan log pengiriman, Danger Notification memiliki notifikasi dan antrean push, dan Event Live Feed memiliki buffer replay dalam mem-

ori. Tidak ada layanan yang membaca tabel milik layanan lain. Domain penyampaian yang memerlukan data hazard memperolehnya dari event, bukan dengan query ke Canonical Store.

5.1.2 Canonical Store milik Aggregator

Canonical Store adalah penyimpanan data hazard hasil normalisasi dan hanya diakses oleh Aggregator. Implementasinya PostgreSQL 17 dengan skema *hybrid*: atribut yang sama untuk semua sumber disimpan sebagai kolom bertipe, sedangkan atribut khusus sumber disimpan dalam satu kolom JSONB.

Tabel 5.2: Struktur Canonical Store

Tabel	Isi dan fungsi
`hazard_events`	Kolom bertipe `hazard_id`, `source`, `source_ref_id`, `hazard_type`, `severity`, `area_name`, koordinat, `occurred_at`, `ingested_at`; kolom `attributes` JSONB. Unique `(source,source_ref_id)` menjaga satu baris per kejadian sumber; indeks pada sumber, jenis, severity, dan `(occurred_at,hazard_id)` untuk filter dan paginasi.
`ingest_watermarks`	Posisi polling, waktu sukses/percobaan terakhir, dan error per sumber; dasar status FRESH/STALE pada P2.
`outbox_events`	Envelope event yang ditulis dalam transaksi yang sama dengan upsert hazard, menunggu dipublikasikan ke broker.
`outbox_dead_letters`	Envelope yang gagal dipublikasikan setelah batas retry.
`volcano_references`	Referensi nama dan koordinat gunung api milik BNPB untuk pemetaan PVMBG.

Ketika PVMBG menambahkan `confidence_level`, nilai tersebut masuk ke `attributes` tanpa `ALTERTABLE`; record lama dan baru tersimpan pada tabel yang sama dan dibaca dengan query yang sama. Filter lintas sumber tetap memakai kolom bertipe dan indeks biasa. Skema diterapkan saat startup pada `services/aggregator/internal/store/postgres.go`.

5.2 Justifikasi dan Alternatif

5.2.1 Batas Layanan

Alternatif pertama adalah **monolit modular**: semua domain dalam satu proses dengan modul terpisah. Rancangan ini lebih sederhana dioperasikan, tetapi kegagalan atau kebocoran memori pada satu modul (misalnya ribuan koneksi feed) menjatuhkan seluruh fungsi, dan satu bahasa harus dipakai untuk semua kebutuhan. Spesifikasi juga meminta satu layanan dapat dihentikan dan di-deploy ulang tanpa mengganggu yang lain.

Alternatif kedua adalah **pemisahan lebih halus**, misalnya satu layanan adapter per sumber (BMKG dan PVMBG) di depan Aggregator. Pemisahan ini memberikan isolasi per sumber, tetapi pemetaan, upsert, dan penulisan outbox perlu berada dalam satu transaksi agar tidak ada event yang hilang. Membagi langkah tersebut ke beberapa layanan membutuhkan koordinasi transaksi terdistribusi. Isolasi per sumber sudah dicapai dengan goroutine dan watermark independen di dalam Aggregator (P2), sehingga pemisahan ini tidak sebanding dengan biayanya.

5.2.2 Penyimpanan Canonical Store

PostgreSQL + JSONB dipilih karena kebutuhan Canonical Store terbagi dua: bagian yang stabil dan sering difilter (sumber, jenis, severity, waktu) serta bagian yang berubah mengikuti sumber. Skema hybrid melayani keduanya dalam satu penyimpanan. Faktor penentu lainnya

Tabel 5.3: Alternatif penyimpanan Canonical Store

Pilihan	**Kelebihan**	**Biaya atau batasan**
PostgreSQL relasional + JSONB (dipilih)	Atribut baru tidak memerlukan migrasi; kolom inti tetap bertipe dan terindeks; transaksi ACID mencakup hazard dan outbox; constraint unik menjaga identitas sumber.	Isi `attributes` tidak divalidasi skema database; query pada atribut dinamis memerlukan operator JSON atau indeks GIN tambahan.
Relasional murni (kolom per atribut)	Semua field bertipe dan tervalidasi database.	Setiap field baru dari sumber memerlukan migrasi `ALTERTABLE` dan rilis ulang; bertentangan dengan kebutuhan menerima `confidence_level` saat runtime.
MongoDB (dokumen)	Skema sepenuhnya fleksibel; payload sumber dapat disimpan apa adanya.	Field inti kehilangan jaminan tipe; constraint unik gabungan, transaksi multi-dokumen hazard–outbox, dan paginasi perlu dirancang ulang. Fleksibilitas yang dibutuhkan sudah dipenuhi JSONB.

adalah transactional outbox pada P5, yang membutuhkan upsert hazard dan penulisan event dalam satu transaksi atomik.

5.3 Batasan Solusi

Pemisahan image dan proses belum membuktikan bahwa setiap layanan dapat direplikasi tanpa perubahan. Publisher outbox belum memakai mekanisme claim/lock antar-worker pada query pengambilan record. Feed memiliki replay dan epoch lokal per proses. SQLite webhook membatasi penulisan serta memerlukan strategi berbagi state sebelum menjalankan beberapa replica. Keterbatasan ini perlu diselesaikan sebelum menyatakan dukungan penskalaan horizontal yang transparan.

Durability Canonical Store bergantung pada volume `aggregator_data` dan prosedur backup; Compose belum menyediakan replikasi atau failover PostgreSQL. Isi `attributes` tidak divalidasi oleh database, sehingga konsistensi tipe atribut dinamis bergantung pada adapter Aggregator.

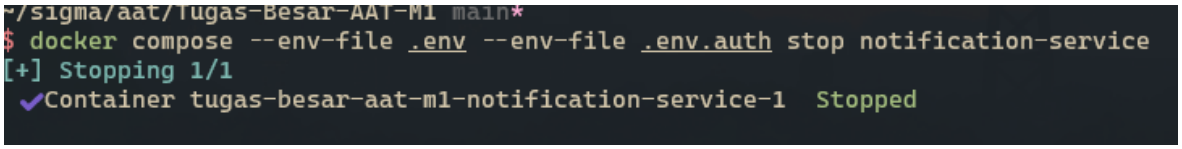
Penyimpanan hazard mempertahankan satu identitas kanonis per referensi sumber. Pemisahan database dan penggunaan constraint tidak dengan sendirinya menyediakan pemrosesan event tepat satu kali.

5.4 Proof-of-Concept dan Keterlacakan

Tangkapan layar pengujian layanan dan penyimpanan pada `docs/tex/evidence/P4/` mencakup rangkaian awal dan tambahan `1-3-new.png` pada commit `4a1973b`. Ringkasan eksekusi otomatis pada `docs/tex/evidence/problem_set_report.md`, dibuat 10 Oktober 2026, mencatat tiga tes P4 lulus, tanpa failure maupun skipped. Cakupan kedua jenis bukti dijelaskan berikut; label lulus tes tidak memperluas observasi melampaui assertion dan screenshot yang tersedia.

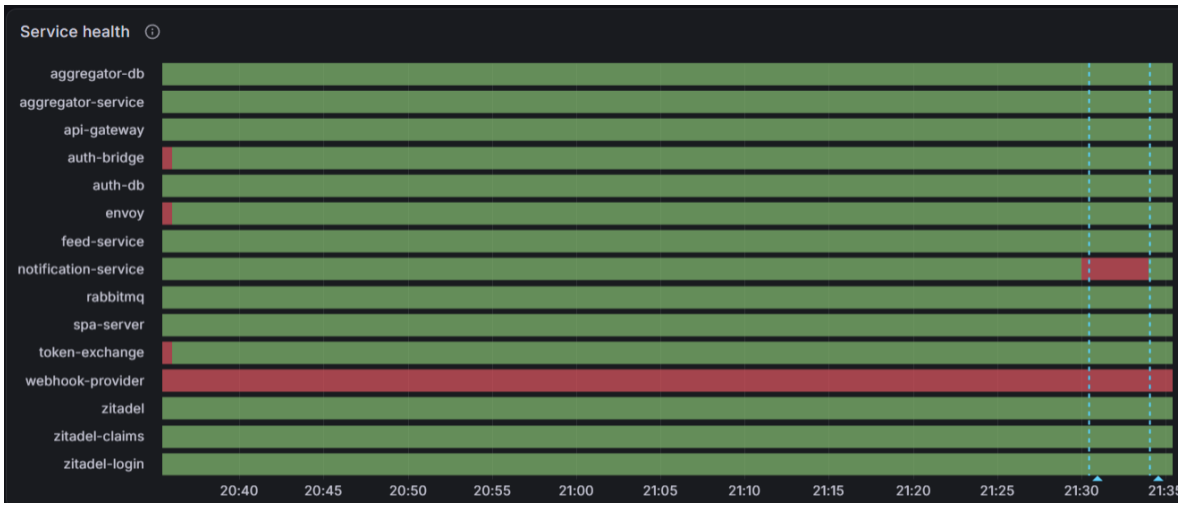
5.4.1 Pemisahan Proses dan Pengoperasian Layanan

Pada pengujian pemisahan proses, operator menghentikan `notification-service`. Halaman notifikasi kemudian menampilkan respons 503 untuk endpoint yang bergantung pada layanan tersebut, sementara feed tetap aktif dan status sumber tetap tersedia. Setelah layanan dibangun dan dijalankan kembali, halaman notifikasi kembali menampilkan data. Urutan ini direkam pada `P4/1-1.png` sampai `P4/1-4.png`.



Gambar 5.1: Penghentian container Danger Notification secara terpisah (`P4/1-1.png`).

Grafik health memperlihatkan interval notification tidak tersedia, sedangkan Aggregator, feed, gateway, dan database tetap terpantau aktif. Webhook memang sedang tidak aktif pada interval grafik tersebut; gambar ini tidak digunakan untuk menyatakan semua layanan hidup bersamaan.



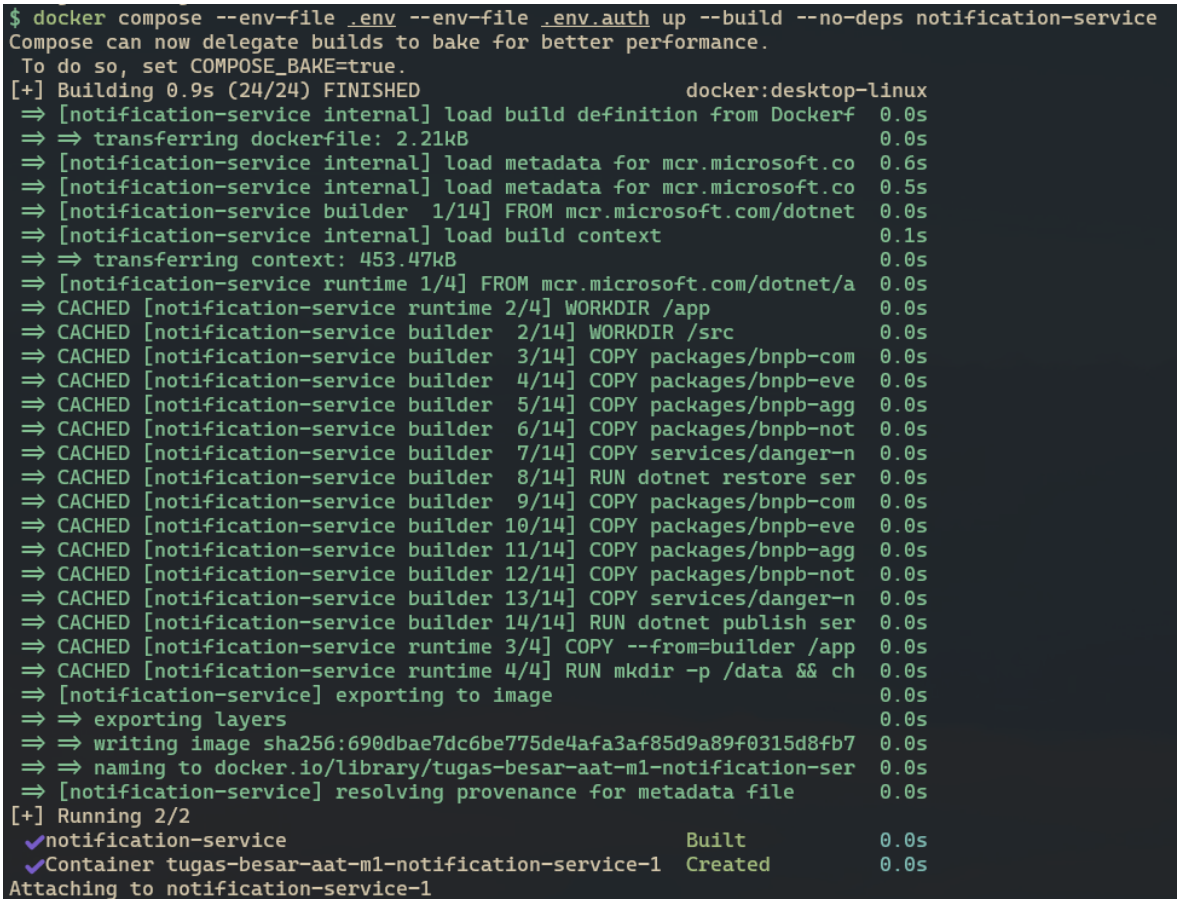
Gambar 5.2: Observasi health ketika notification dihentikan dan dipulihkan (`P4/1-5.png`).

Tes otomatis `tests/test_problem_4.py` melengkapi observasi dengan menghentikan satu layanan, meminta hazard BMKG melalui API, memeriksa respons 200 beserta data, lalu menjalankan kembali layanan. Ini mendukung ketersediaan jalur baca independen selama penghentian yang diuji. Bukti tambahan `P4/1-3-new.png` memakai perintah berikut:

```
docker compose --env-file .env --env-file .env.auth up --build --no-deps notification-service
```

Output memperlihatkan 24 langkah build hanya untuk Notification, image berstatus Built, dan container Notification berstatus Created lalu di-attach. Sebagian besar langkah memakai build cache. Opsi `--no-deps` membatasi operasi pada target sehingga dependency tidak ikut dibangun atau dijalankan oleh perintah ini. Bukti ini menggantikan `P4/1-3.png` sebagai rujukan build satu layanan; screenshot lama tetap tersimpan sebagai riwayat.

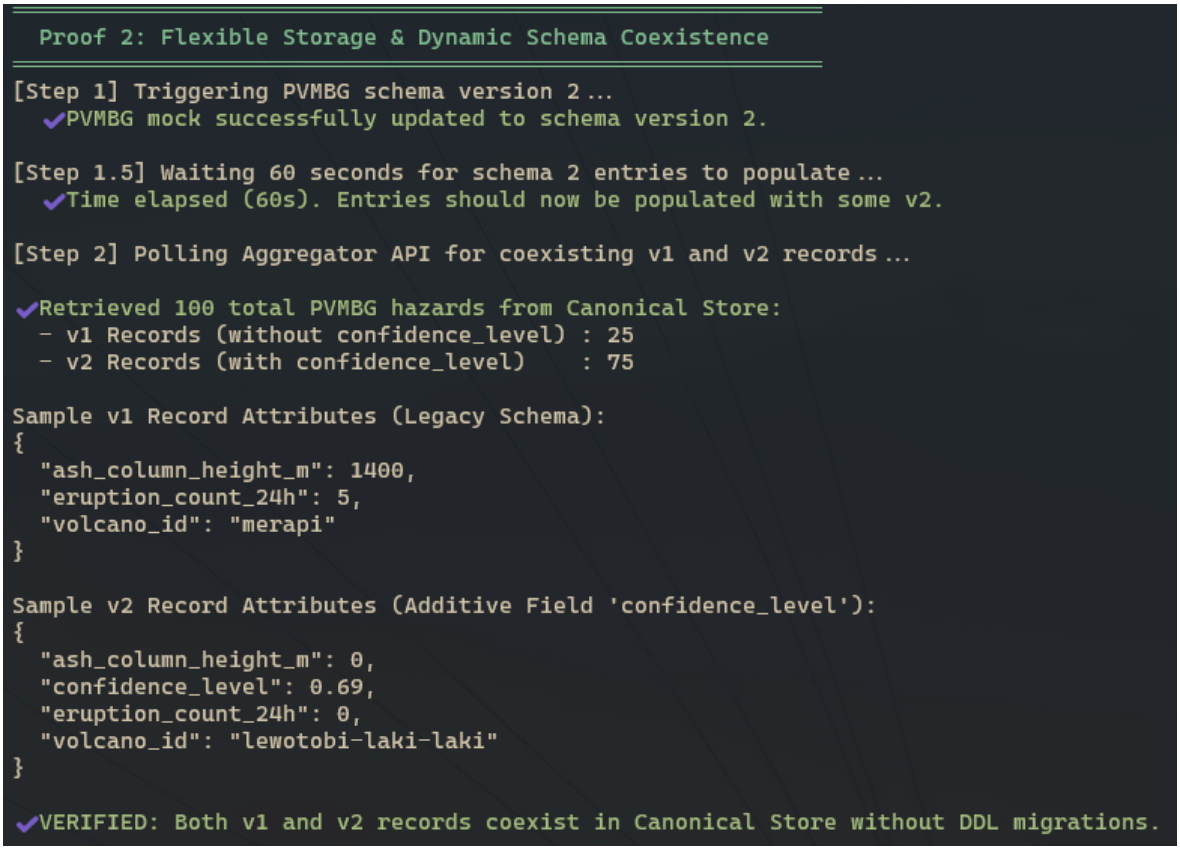
Grafik health dan pembacaan API pada rangkaian awal melengkapi observasi ketersediaan jalur independen. Screenshot build baru tidak memuat perbandingan ID/start time seluruh container atau health setelah attach, sehingga tidak digunakan sebagai catatan lengkap riwayat restart setiap container di luar interval yang direkam.



Gambar 5.3: Build dan pembuatan container Notification dengan `--no-deps`; output hanya menampilkan layanan target (`P4/1-3-new.png`).

5.4.2 Atribut Dinamis dan Koeksistensi Skema

Setelah kontrol PVMBG diubah ke versi 2 dan ditunggu 60 detik, pembacaan 100 hazard PVMBG melalui Canonical Store menemukan 25 record tanpa `confidence_level` dan 75 record dengan atribut tersebut. Screenshot menampilkan contoh kedua bentuk atribut, termasuk nilai confidence 0,69. Pemeriksaan tabel memperlihatkan bahwa `attributes` bertipe JSONB; `confidence_level` tidak menjadi kolom terpisah.



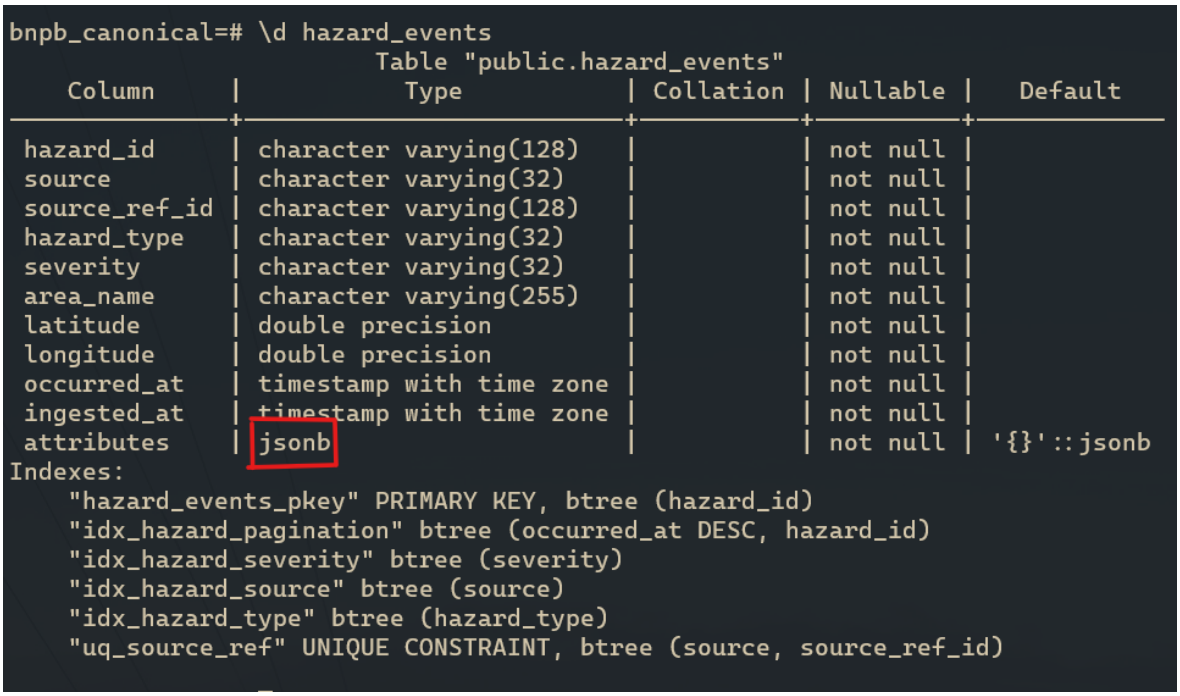
Gambar 5.4: Record PVMBG sebelum dan sesudah penambahan atribut tersedia bersamaan melalui API (`P4/2-1.png`).

Hasil ini mendukung rancangan penyimpanan fleksibel dan koeksistensi data v1/v2. Tes otomatis memeriksa keberadaan kedua kelompok record serta tipe numerik confidence. Screenshot DDL yang tersedia adalah satu snapshot struktur, bukan pasangan hasil sebelum/sesudah. Oleh sebab itu, tidak ada angka perubahan DDL atau hasil perbandingan hash schema yang diklaim.

5.4.3 Kepemilikan Data dan Batas Pengujian

Tangkapan layar `P4/3.png` memperlihatkan inspeksi konfigurasi serta keberhasilan pembacaan status sumber melalui API. Pada kode aplikasi, operasi Canonical Store ditangani Aggregator; consumer memperoleh data lewat event atau API. Pemeriksaan otomatis mencari variabel `CANONICAL_STORE_DSN` secara literal dalam blok Compose layanan lain. Pemeriksaan ini menunjukkan konvensi konfigurasi yang diperiksa, tetapi tidak membuktikan pembatasan akses database pada tingkat jaringan atau hak pengguna.

Compose tidak lagi menyuntikkan seluruh `.env` ke container; setiap layanan menerima variabel yang diperlukannya saja, dan DSN Canonical Store hanya diberikan kepada Aggregator. Namun, database berada pada jaringan bersama dan port PostgreSQL diekspos ke host; collector observabilitas mempunyai koneksi PostgreSQL untuk metrik. Dengan demikian, istilah kepemi-



Gambar 5.5: Struktur tabel hazard menggunakan JSONB untuk atribut sumber (`P4/2-2.png`).

likan pada laporan ini berarti pemisahan tanggung jawab aplikasi. Pembatasan credential per layanan, role metrik khusus, dan jaringan database perlu diterapkan bila kriteria akses eksklusif hendak dijamin secara teknis.

Tabel 5.4: Cakupan bukti P4 dan batas interpretasi

Objek	**Hasil dan batas bukti**
Independensi proses	Stop dan pemulihan diamati; bukti baru memperlihatkan build target dengan `--no-deps`. Health jalur lain tersedia pada rangkaian awal. Identitas semua container belum dibandingkan.
Atribut dinamis	25 record tanpa confidence dan 75 dengan confidence terbaca; kolom JSONB diperlihatkan.
Kepemilikan Canonical Store	Jalur aplikasi melalui Aggregator/event ditunjukkan; konfigurasi belum menjamin isolasi credential dan jaringan.
Persistensi lintas restart	Volume PostgreSQL, webhook, dan notification tersedia pada kode. Tangkapan layar pengujian layanan dan penyimpanan tidak membandingkan ID/jumlah data ketiganya sebelum dan sesudah restart.

Hasil historis pengujian container mock pada `docs/tex/evidence/mock-container-2026-09-30.json` tetap hanya berlaku untuk persistensi database mock yang ketika itu diuji; tidak digunakan sebagai pengganti pengujian volume Canonical Store, webhook, atau notification. Pengujian unit Aggregator pada `docs/tex/evidence/aggregator-unit-2026-10-09.jsonl` mendukung transformasi atribut, sedangkan ringkasan eksekusi otomatis dan tangkapan layar pengujian layanan dan penyimpanan menambahkan pengamatan pada stack berjalan. Sumber bukti terdaftar pada `docs/tex/evidence/manifest.json`; petunjuk pengujian tersedia pada `docs/e2e-testing.md`.

Bab 6

P5: Distribusi Event

6.1 Rancangan Solusi

Aggregator memublikasikan perubahan hazard melalui exchange RabbitMQ bertipe fanout. Setiap consumer memiliki queue sendiri, sehingga satu publication disalin ke Event Live Feed, Webhook Provider, dan Danger Notification. Ketiga layanan dapat memproses publication yang sama untuk tujuan berbeda. Menempatkan ketiganya pada satu queue akan menghasilkan pembagian pekerjaan antar-consumer, bukan penyalinan event ke seluruh layanan.

Tabel 6.1: Topologi distribusi event

Komponen	**Nama contoh/default**	**Peran**
Exchange	`bnpb.hazards.v1`	Fanout durable.
Queue feed	`bnpb.event-live-feed.v1`	Event streaming dan replay.
Queue webhook	`bnpb.webhook-provider.v1`	Callback subscription.
Queue notification	`bnpb.danger-notification.v1`	Push SIAGA/AWAS.
DLQ per consumer	Nama layanan dengan akhiran `.dlq.v1`	Menampung pesan yang ditolak tanpa requeue.

Nama exchange dan queue berasal dari konfigurasi. Producer hanya mendeklarasikan exchange; tiap consumer mendeklarasikan queue durable, dead-letter queue, dan binding miliknya. Routing dead-letter menggunakan default exchange dengan routing key nama DLQ. Ketiga consumer menggunakan prefetch satu pesan dan acknowledgement manual. Pengaturan ini membatasi pesan yang belum diakui, tetapi juga membuat pekerjaan lambat dalam satu consumer menahan pekerjaan berikutnya pada consumer tersebut.

Publication memakai envelope Protobuf `HazardEventPublished`, berisi `event_id`, `schema_version`, sumber, publisher, correlation ID, waktu publication, dan payload hazard kanonis. Implementasi saat ini menggunakan versi schema 1 dan publisher `aggregator`. `event_id` berupa UUID publication dan berbeda dari `hazard_id` yang stabil untuk satu referensi sumber. Pembentukan envelope dapat ditelusuri pada `services/aggregator/internal/ingest/mapper.go`.

Alur penyimpanan dan publication menggunakan pola transactional outbox:

1. Poller memetakan data sumber ke hazard kanonis, lalu membuka transaksi PostgreSQL.
2. Upsert hazard dan penyisipan envelope ke `outbox_events` menggunakan transaksi yang sama, kemudian transaksi di-commit. Envelope hanya dibuat untuk hazard baru atau hazard

yang kolom inti maupun `attributes`-nya berubah; pengambilan ulang record yang sama akibat watermark inklusif tidak menghasilkan publication baru. Bila pembentukan atau penyisipan envelope gagal, putaran polling dihentikan tanpa commit dan watermark tidak dimajukan.
3. Publisher mengambil paling banyak 20 publication pending setiap 200 ms dan mengirim pesan persisten ke exchange.
4. Publisher menunggu publisher confirm dari broker sebelum mengisi `published_at`. Kegagalan publish menaikkan retry record.
5. Pada batas retry, contoh konfigurasi lima percobaan, record ditandai gagal dan disalin ke tabel `outbox_dead_letters`.

Implementasi publisher terdapat pada `services/aggregator/internal/broker/publisher.go` dan `services/aggregator/internal/store/outbox_store.go`. Tabel dead-letter outbox merupakan state PostgreSQL producer, berbeda dari DLQ RabbitMQ masing-masing consumer. Kegagalan koneksi awal broker diulang setiap dua detik; retry koneksi tersebut bukan penghitung percobaan publish setiap record.

Setiap consumer menentukan titik acknowledgement sesuai pekerjaannya:

- **Event Live Feed** memvalidasi envelope, menambahkan event ke replay buffer, lalu ack. ID yang masih berada dalam replay window dideduplikasi. Envelope rusak atau tidak didukung langsung di-nack tanpa requeue. Koneksi diulang dengan backoff 1–30 detik.
- **Webhook Provider** mencari subscription yang sesuai filter, memeriksa dedup, membentuk payload sesuai view, dan mengirim HTTP POST dengan tanda tangan HMAC-SHA256 pada `X-BNPB-Signature`. Hasil dicatat ke SQLite dan delivery sukses ditandai untuk dedup. Default dispatcher melakukan lima percobaan dengan timeout 10 detik per request serta jeda 1, 2, 4, dan 8 detik. Delivery yang tetap gagal setelah seluruh percobaan dicatat sebagai `FAILED` dan pesan broker tetap di-ack, sehingga endpoint yang mati tidak memicu requeue tanpa akhir. Error internal, misalnya gagal membaca subscription atau menulis log, memperoleh satu redelivery sebelum ditolak menuju DLQ.
- **Danger Notification** memproses envelope dan memilih severity SIAGA/AWAS. Event yang memenuhi filter disimpan secara transaksional pada SQLite sebagai notifikasi dan pekerjaan push pending, kemudian pesan broker di-ack. Event di luar filter di-ack tanpa pekerjaan push. `PushOutboxWorker` mengirim melalui FCM secara terpisah dan menyimpan hasil pengiriman beserta jadwal retry. Parse error Protobuf langsung ditolak; error pemrosesan consumer lainnya memperoleh satu redelivery sebelum ditolak menuju DLQ. Ack broker tidak menunggu keberhasilan FCM atau penerimaan pada perangkat.

6.1.1 Penanganan At-Least-Once: Idempotent Consumer

Sistem memilih semantik pengiriman **at-least-once** dan menangani duplikasi yang ditimbulkannya dengan pola **idempotent consumer** [11]. At-most-once (ack sebelum diproses) ditolak karena event bahaya dapat hilang ketika consumer crash. Exactly-once ditolak karena RabbitMQ tidak menyediakannya dan efek samping consumer (HTTP callback, push FCM) berada di luar transaksi broker. Konsekuensinya, satu publication dapat diterima consumer lebih dari sekali dari dua sumber berikut:

1. **Redelivery oleh broker**: consumer crash atau koneksi putus sebelum ack, atau consumer melakukan nack dengan requeue.
2. **Publikasi ulang oleh outbox**: publisher crash setelah broker mengonfirmasi pesan tetapi sebelum `published_at` tersimpan, sehingga record yang sama dikirim kembali.

Kedua kasus membawa `event_id` yang sama, karena ID tersebut dibuat sekali ketika envelope

ditulis ke outbox dan tidak dibuat ulang saat pengiriman. Karena itu `event_id` dipakai sebagai *kunci idempotensi*. Setiap consumer menerapkan tiga aturan yang sama: (1) periksa apakah `event_id` sudah diproses, (2) simpan efek dan penanda "sudah diproses" sebelum ack, (3) ack hanya setelah keduanya tersimpan. Pesan yang diterima ulang kemudian dikenali dan tidak menghasilkan efek bisnis kedua.

Tabel 6.2: Penerapan idempotent consumer pada tiap layanan

Consumer	**Kunci dan penyimpanan**	**Perilaku saat pesan diterima ulang**	**Titik ack**
Event Live Feed	`event_id` dalam himpunan `seen` di memori, sepanjang jendela replay.	Event yang sudah ada tidak ditambahkan ke buffer dan tidak disiarkan ulang ke subscriber.	Setelah event masuk buffer replay.
Webhook Provider	Pasangan `(event_id,webhook_id)` pada tabel SQLite `delivery_dedup`.	Subscription yang sudah sukses dilewati; hanya subscription yang belum sukses dikirim ulang. Kegagalan HTTP setelah retry dispatcher dicatat `FAILED` tanpa requeue; error penyimpanan internal memperoleh satu redelivery lalu menuju DLQ.	Setelah setiap percobaan delivery selesai dan log tercatat.
Danger Notification	`event_id` sebagai primary key tabel `notifications` di SQLite.	Record yang sudah ada hanya digabung; status push hanya bergerak maju (Pending, Failed, Delivered), sehingga tidak ada pekerjaan push baru. Revisi lama yang datang terlambat tidak menggantikan revisi current.	Setelah transaksi SQLite commit.

Selain deduplikasi per `event_id`, Notification membedakan *revisi* dari *duplikat*. Publication baru untuk `hazard_id` yang sama mempunyai `event_id` berbeda dan diperlakukan sebagai revisi: revisi sebelumnya ditandai tidak current. Urutan revisi ditentukan dari waktu publication, sehingga pesan yang tiba tidak berurutan tidak menimpa data yang lebih baru. Di sisi produsen, Aggregator juga idempoten terhadap pengambilan ulang data sumber: upsert dengan kunci unik `(source,source_ref_id)` menjaga satu baris hazard per kejadian sumber.

Kode consumer berada pada `services/event-live-feed/src/adapters/inbound/rabbitmq.rs`, `services/webhook-provider/internal/consumer/consumer.go`, dan `services/danger-notification/BNPB.Notification.Server/Adapters/Inbound/RabbitMqConsumer.cs`. Header trace dibawa melalui AMQP untuk menghubungkan span publish dan consume. Identitas publication membantu menelusuri satu event di beberapa consumer, sementara correlation ID menghubungkannya dengan putaran ingestion.

6.2 Justifikasi dan Alternatif

Fanout dipilih karena semua consumer menerima event hazard yang sama lalu melakukan filter atau proyeksi sesuai fungsi layanan. Producer tidak perlu mengetahui alamat callback maupun perangkat pengguna. Queue terpisah memungkinkan consumer yang berhenti menimbun backlog tanpa memaksa consumer lain berhenti memproses queue masing-masing, selama broker dan resource bersama masih tersedia.

Alternatif panggilan HTTP langsung dari Aggregator ke setiap consumer mengurangi kebutuhan broker, tetapi menjadikan publisher bertanggung jawab atas timeout, retry, dan availability setiap tujuan. Alternatif exchange topic dapat mengurangi lalu lintas dengan routing menurut

sumber atau jenis hazard; konsekuensinya ialah penambahan aturan routing dan kesepakatan key. Fanout mempertahankan topologi yang sederhana untuk tiga consumer saat ini.

Alternatif log terdistribusi berbasis offset memberi retensi dan replay yang lebih panjang, tetapi memerlukan pengelolaan partition, offset, dan consumer group. Kebutuhan milestone dapat ditangani RabbitMQ dan outbox PostgreSQL. Keputusan ini tidak menyamakan queue dengan arsip event permanen: event yang telah di-ack tidak lagi menjadi backlog broker, dan replay feed memiliki retensi sendiri.

6.3 Batasan Solusi

Pada transport broker, persistent message, publisher confirm, manual ack, dan redelivery mendukung pola *at-least-once* selama queue sudah terikat, state broker masih tersedia, dan pesan belum dihentikan oleh kebijakan retry atau DLQ. Confirm producer dan ack consumer mengonfirmasi tahap berbeda [7]. Jaminan tersebut tidak sama dengan keberhasilan efek bisnis untuk setiap subscriber. Semantik sistem belum exactly-once. Publisher confirm dan penandaan `published_at` merupakan dua operasi berbeda, sehingga crash di antaranya dapat menyebabkan publication dikirim ulang. Demikian pula, keberhasilan HTTP/FCM dan acknowledgement broker tidak berada dalam transaksi yang sama. Mekanisme dedup harus dinilai per consumer:

1. Feed hanya mendeduplikasi sepanjang replay window dalam memori. Ack setelah append tidak berarti data tersimpan di disk; restart menghapus replay dan himpunan dedup.
2. Webhook menyimpan dedup persisten per pasangan publication dan subscription, tetapi crash setelah penerima menerima POST dan sebelum penandaan dedup masih dapat menghasilkan pengiriman ulang. Penerima juga perlu menangani duplikasi. Sebaliknya, delivery yang berstatus `FAILED` setelah retry dispatcher tidak dikirim ulang otomatis karena pesan broker sudah di-ack.
3. Notification memakai ID publication sebagai primary key SQLite. Pengiriman ulang ID yang sama tidak membuat notifikasi baru, dan pembaruan status mempertahankan hasil push yang sudah delivered. Revisi baru suatu hazard mempunyai ID publication berbeda dan menandai revisi sebelumnya sebagai tidak current. Lease outbox membantu koordinasi pengiriman, tetapi crash setelah FCM menerima pesan dan sebelum hasil tersimpan tetap dapat menghasilkan push berulang. Dedup penyimpanan tidak sama dengan exactly-once pada perangkat pengguna.

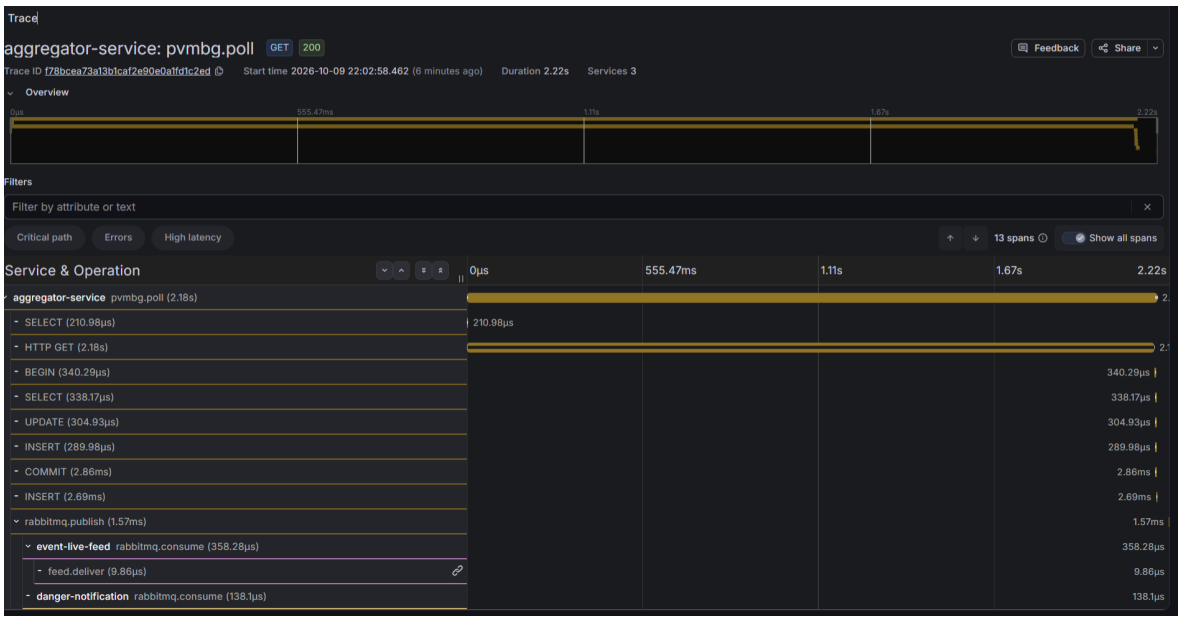
Publication memakai `mandatory=false`; apabila queue consumer belum pernah dibuat dan diikat ketika pesan dipublikasikan, publisher confirm tidak membuktikan consumer tersebut memperoleh salinannya. State broker disimpan pada volume `rabbitmq_data`, sehingga queue durable dan pesan persisten dapat bertahan ketika container broker dibuat ulang selama volume tersebut dipertahankan.

6.4 Proof-of-Concept dan Keterlacakan

Tangkapan layar pengujian pada `docs/tex/evidence/P5/` merekam distribusi event, penghentian consumer, pemulihan backlog, dan pengoperasian consumer tambahan tanggal 9 Oktober 2026. Ringkasan eksekusi otomatis pada `docs/tex/evidence/problem_set_report.md` mencatat empat tes P5 lulus tanpa failure atau skipped pada 10 Oktober 2026. Uraian berikut mengutamakan kondisi yang benar-benar terlihat pada artefak; screenshot beberapa sesi tidak diperlakukan sebagai satu rangkaian waktu yang utuh.

6.4.1 Publication dan Dua Consumer Independen

Trace pengujian distribusi event memperlihatkan polling PVMBG oleh Aggregator, transaksi penyimpanan, span `rabbitmq.publish`, serta span `rabbitmq.consume` pada Event Live Feed dan Danger Notification. Keterhubungan span dalam trace yang sama mendukung distribusi satu publication ke dua consumer, tanpa panggilan langsung producer ke endpoint masing-masing.



Gambar 6.1: Trace publication Aggregator dengan cabang konsumsi pada Feed dan Notification (`P5/1.png`).

Screenshot `P5/2-1.png` menunjukkan exchange `bnpb.hazards.v1` bertipe fanout dan binding queue Feed serta Notification. Pada sesi tersebut Webhook Provider belum aktif. `P5/2-2.png` dan `P5/2-3.png` memperlihatkan dua queue utama beserta DLQ, consumer aktif, dan laju delivery/ack pada masing-masing queue. Bukti ini memenuhi demonstrasi minimal dua consumer; tidak mengklaim ketiga layanan telah memproses satu ID publication yang terlihat lengkap pada screenshot yang sama.

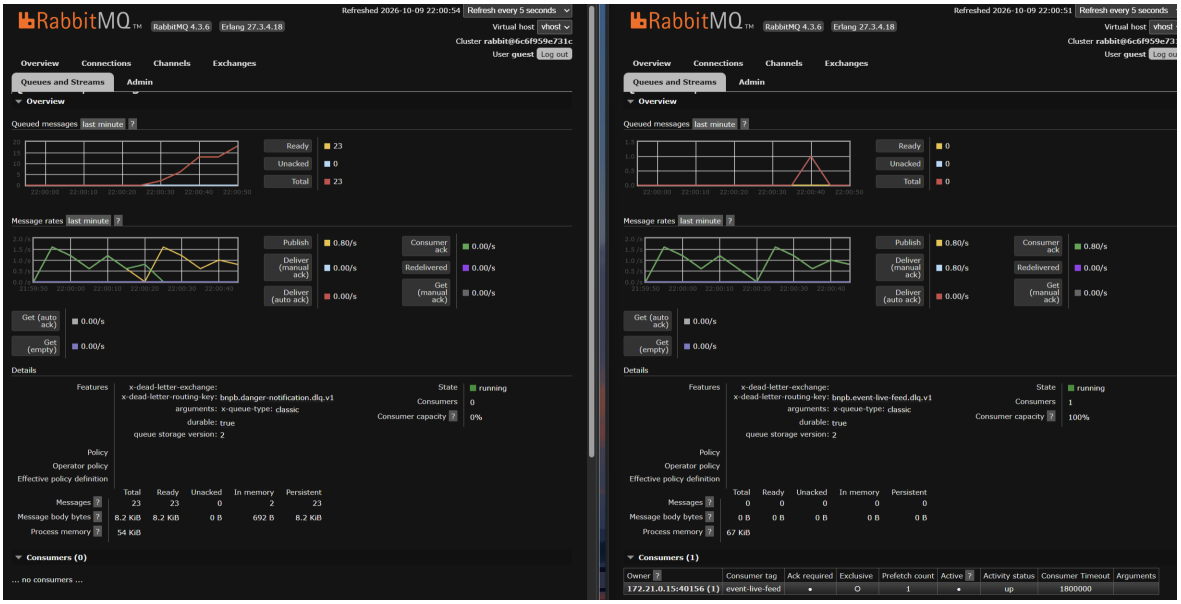
Tes otomatis membuka SSE dan menerima `hazard.upsert`, lalu memeriksa keberadaan queue Notification. Ringkasan eksekusi otomatis menyebut Feed dan Webhook, sedangkan implementasi tes `test_02` menggunakan Notification. Pasangan yang dipakai pada pembahasan hasil ini adalah Feed–Notification, sesuai kode dan trace pengujian distribusi event. Assertion queue tes tersebut belum mencocokkan ID publication; trace manual menjadi bukti tambahan untuk hubungan publish–consume.

6.4.2 Consumer Berhenti dan Pemulihan Backlog

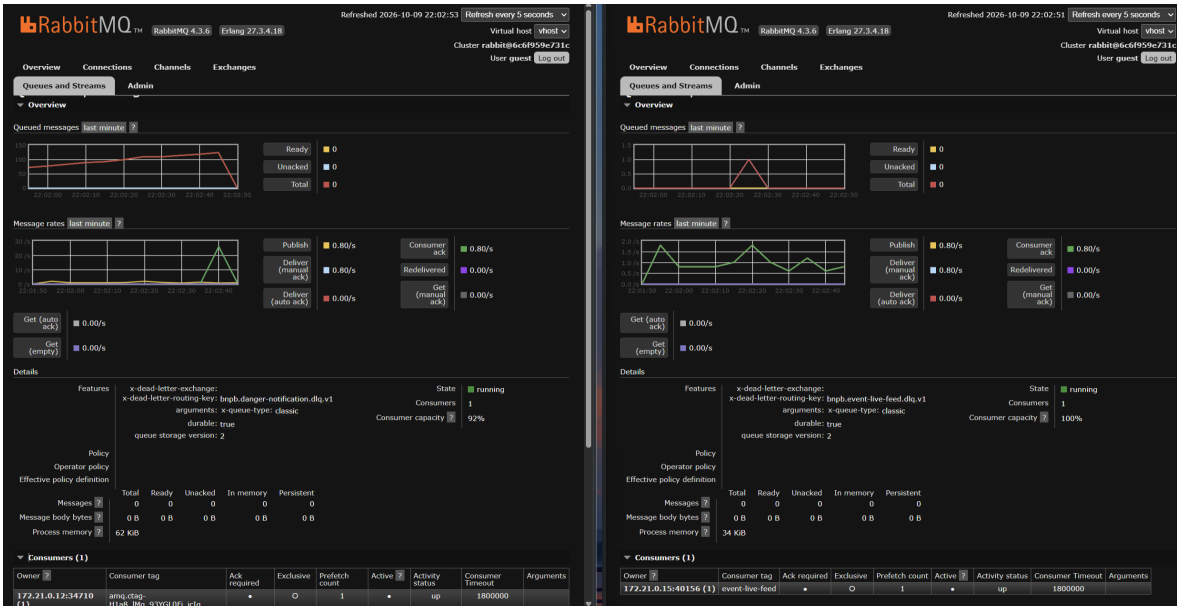
Ketika Notification dihentikan, queue miliknya menunjukkan nol consumer dan 23 pesan ready, sementara queue Feed tetap mempunyai satu consumer serta laju delivery dan ack 0,80 pesan/detik. Broker masih menerima publication untuk kedua queue. Kondisi ini menunjukkan bahwa tidak aktifnya satu consumer tidak menghentikan distribusi ke consumer lainnya.

Setelah Notification kembali aktif, grafik queue memperlihatkan backlog yang sebelumnya mencapai lebih dari 100 pesan turun menjadi nol. Jumlah consumer kembali satu dan terdapat lonjakan delivery/ack saat pengurasan. Feed tetap aktif selama interval yang diperlihatkan.

Event yang terbit ketika consumer berhenti tersimpan sebagai pesan ready pada queue yang



Gambar 6.2: Backlog Notification bertambah saat consumernya berhenti, sedangkan Feed tetap memproses event (`P5/3-1.png`).



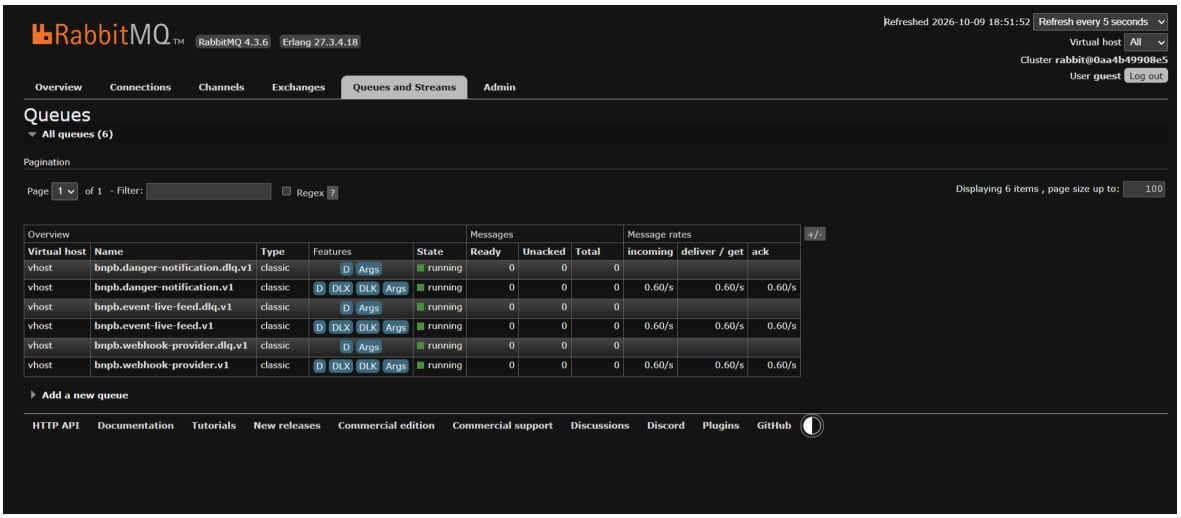
Gambar 6.3: Backlog Notification terkuras setelah consumer kembali aktif (`P5/3-2.png`).

sudah terikat. Ketika consumer kembali, pesan tersebut baru dikirim dan di-ack; mekanisme ini berbeda dari redelivery pesan yang sudah pernah dikirim tetapi belum di-ack. Pada screenshot laju redelivery bernilai nol, sehingga hasil tersebut tidak digunakan untuk mengklaim pengujian redelivery atau DLQ. Berkurangnya backlog membuktikan pemrosesan broker, bukan jumlah push FCM yang berhasil tampil pada perangkat.

Tes otomatis P5 juga menghentikan Feed dan memeriksa pembacaan hazard serta kembalinya endpoint SSE. Tes itu belum mengassert kenaikan dan pengurasan backlog secara numerik. Bukti backlog pada laporan berasal dari gambar manual Notification di atas, bukan dari perluasan arti status PASS pada ringkasan otomatis.

6.4.3 Consumer Tambahan pada Exchange yang Sama

Pada sesi pengujian lainnya, Webhook Provider dijalankan ketika producer dan layanan lain sudah tersedia. Log `P5/4-1.png` memperlihatkan pembuatan container/volume webhook, inisialisasi SQLite, serta koneksi consumer ke queue `bnpb.webhook-provider.v1`. Tangkapan layar broker kemudian memperlihatkan queue webhook bersama Feed dan Notification; ketiga queue utama mempunyai laju incoming, delivery, dan ack.

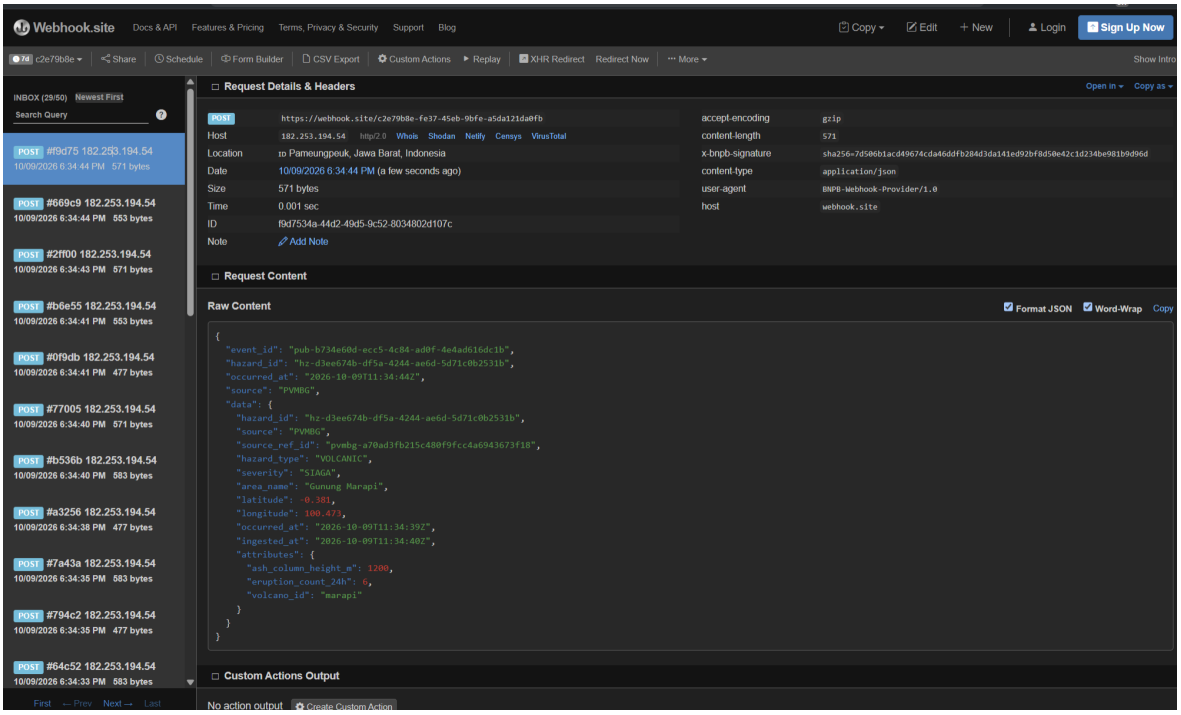


Gambar 6.4: Tiga queue consumer aktif setelah Webhook Provider tersedia (`P5/4-4.png`).

Callback HTTP yang diterima memuat `event_id`, `hazard_id`, sumber PVMBG, dan snapshot hazard. Hal ini menunjukkan bahwa jalur consumer webhook menghasilkan callback data, bukan sekadar queue yang berhasil dibuat. Endpoint callback eksternal pada bukti hanya digunakan untuk demo.

Bukti alternatif merekam penambahan queue baru dan binding ke exchange fanout melalui Management API. Screenshot binding sendiri belum menunjukkan pengambilan pesan oleh consumer. Tes otomatis `test_04_dynamic_new_consumer_addition_zero_touch` melengkapi bagian ini dengan membuat queue unik, mengikatnya ke exchange, menunggu serta mengambil satu pesan, lalu menghapus queue uji. Operasi itu tidak mengubah kode producer. Bukti yang tersedia tidak menyimpan perbandingan ID/start time container Aggregator pada seluruh sesi, sehingga tidak digunakan untuk menyimpulkan riwayat restart producer di luar interval observasi.

Pengujian DLQ, injeksi publication duplikat, pemulihan setelah broker dibuat ulang, dan keberhasilan push pada perangkat tidak direkam dalam rangkaian pengujian distribusi event ini. Batasan at-least-once, retensi replay Feed, dan efek samping FCM yang dijelaskan sebelumnya tetap berlaku. Ringkasan eksekusi otomatis memuat hasil tes; rincian klaim harus dibaca



Gambar 6.5: Payload hazard diterima pada endpoint callback Webhook Provider (`P5/4-5.png`).

Tabel 6.3: Ringkasan bukti P5

Kriteria	**Hasil yang didukung artefak**
Publication melalui broker	Trace publish–consume dan exchange fanout diperlihatkan.
Dua consumer independen	Feed dan Notification menerima melalui queue masing-masing dan terhubung pada trace publication.
Consumer offline/pulih	Notification nol consumer dengan backlog 23; sesudah aktif, backlog yang sempat melebihi 100 turun menjadi nol; Feed tetap aktif.
Consumer tambahan	Webhook memiliki queue dan menerima callback; tes queue dinamis juga mengambil pesan tanpa perubahan kode producer.

bersama assertion pada `tests/test_problem_5.py` dan gambar manual, sebagaimana inventaris pada `docs/tex/evidence/manifest.json`.

Daftar Pustaka

[1] Tim Pengampu IF4031. *IF4031 – Tugas Besar – M1*. Dokumen spesifikasi tugas yang diberikan, bagian P1–P5 dan ketentuan laporan.

[2] Kelompok nasigoreng. *Tugas-Besar-AAT-M1*, snapshot repository commit `122ca7b`, 10 Oktober 2026; artefak bukti berasal dari commit `4a1973b`. Kontrak pada `contracts/`, implementasi pada `services/`, orkestrasi pada `compose.yaml`, serta artefak pengujian pada `docs/tex/evidence/`.

[3] OpenID Foundation. *OpenID Connect Core 1.0 incorporating errata set 2*. `https://openid.net/specs/openid-connect-core-1_0.html`.

[4] N. Sakimura, J. Bradley, dan N. Agarwal. *RFC 7636: Proof Key for Code Exchange by OAuth Public Clients*. 2015. `https://www.rfc-editor.org/info/rfc7636/`.

[5] B. Campbell dkk. *RFC 8693: OAuth 2.0 Token Exchange*. 2020. `https://www.rfc-editor.org/info/rfc8693/`.

[6] PostgreSQL Global Development Group. *PostgreSQL Documentation: JSON Types*. `https://www.postgresql.org/docs/current/datatype-json.html`. Referensi konsep JSONB; container implementasi menggunakan PostgreSQL 17.

[7] RabbitMQ. *Consumer Acknowledgements and Publisher Confirms*. `https://www.rabbitmq.com/docs/confirms`.

[8] M. Fowler. *TolerantReader*. 2011. `https://martinfowler.com/bliki/TolerantReader.html`.

[9] Grafana Labs. *Grafana k6 Documentation: Scenarios and Executors*. `https://grafana.com/docs/k6/latest/using-k6/scenarios/`.

[10] C. Richardson. *Pattern: Database per service*. `https://microservices.io/patterns/data/database-per-service.html`.

[11] C. Richardson. *Pattern: Idempotent Consumer*. `https://microservices.io/patterns/communication-style/idempotent-consumer.html`.